



**VietNam National University
University of Engineering and Technology**

EMBEDDED SYSTEM DEVELOPMENT

(PHÁT TRIỂN HỆ THỐNG NHÚNG)

TS. Nguyễn Kiêm Hùng

Email: kiemhung@vnu.edu.vn

Objectives

In this lecture you will be introduced to:

- Serial communications: Concepts, Tools and Software
- SPI communications
- I²C communications
- UART communications

Outline

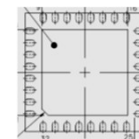
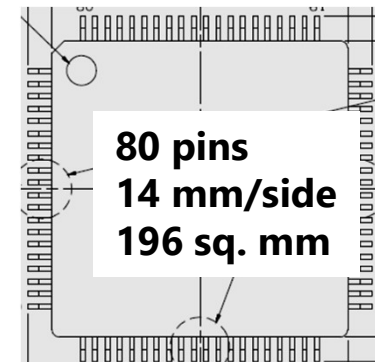


- **Serial communication Concept**
- Software Architecture for Handling asynchronous Communication
- SPI Communications
- I2C Communications
- Asynchronous serial (UART) Communications
- Protocol Comparison
- Summary

Why Communicate Serially?

- Although native word size for CPU is 32 bits, sending all of a word's bits simultaneously has disadvantages:
 - **Cost and weight:** larger IC package, more wires, larger connectors
 - **Mechanical reliability:** more wires => more connector contacts to fail
 - **Timing complexity:** some bits may arrive later than others due to variations in capacitance and resistance across conductors
 - **Circuit complexity and power:** may not want to have 16 different transmitters + receivers in the system
- Communicating serially reduces number of signals needed

Shrinking Packages for Freescale MCUs

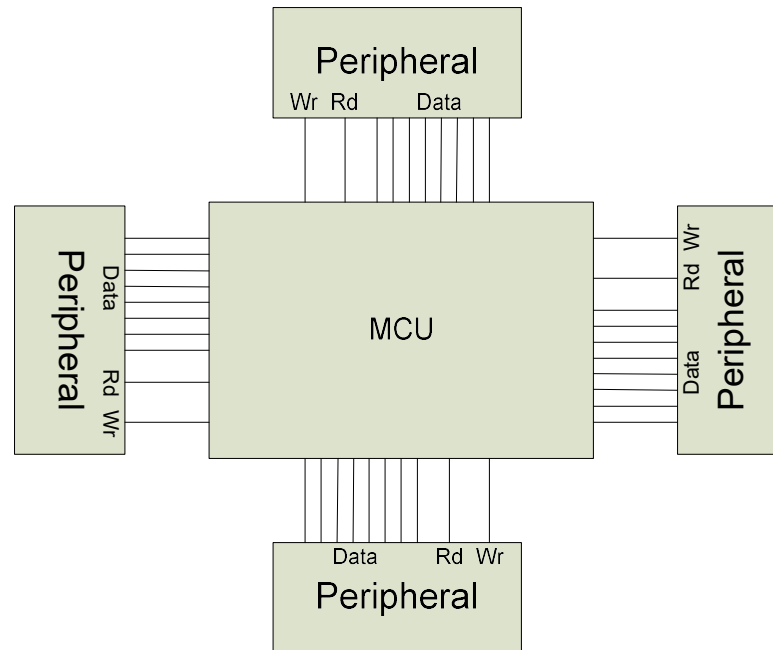


32 pins
5 mm/side
25 sq. mm



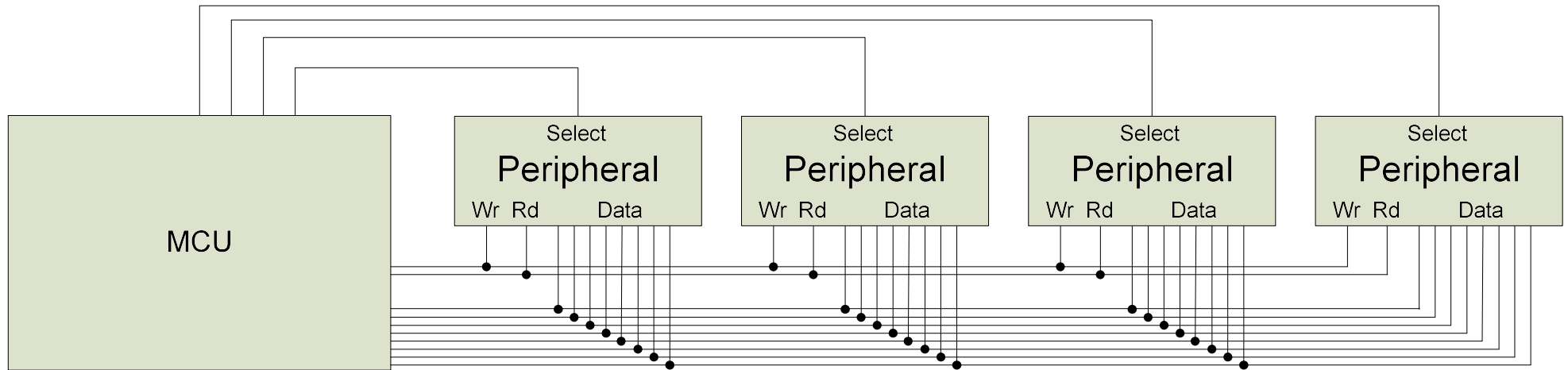
20 pins
1.94 mm/side
3.76 sq. mm

Example System



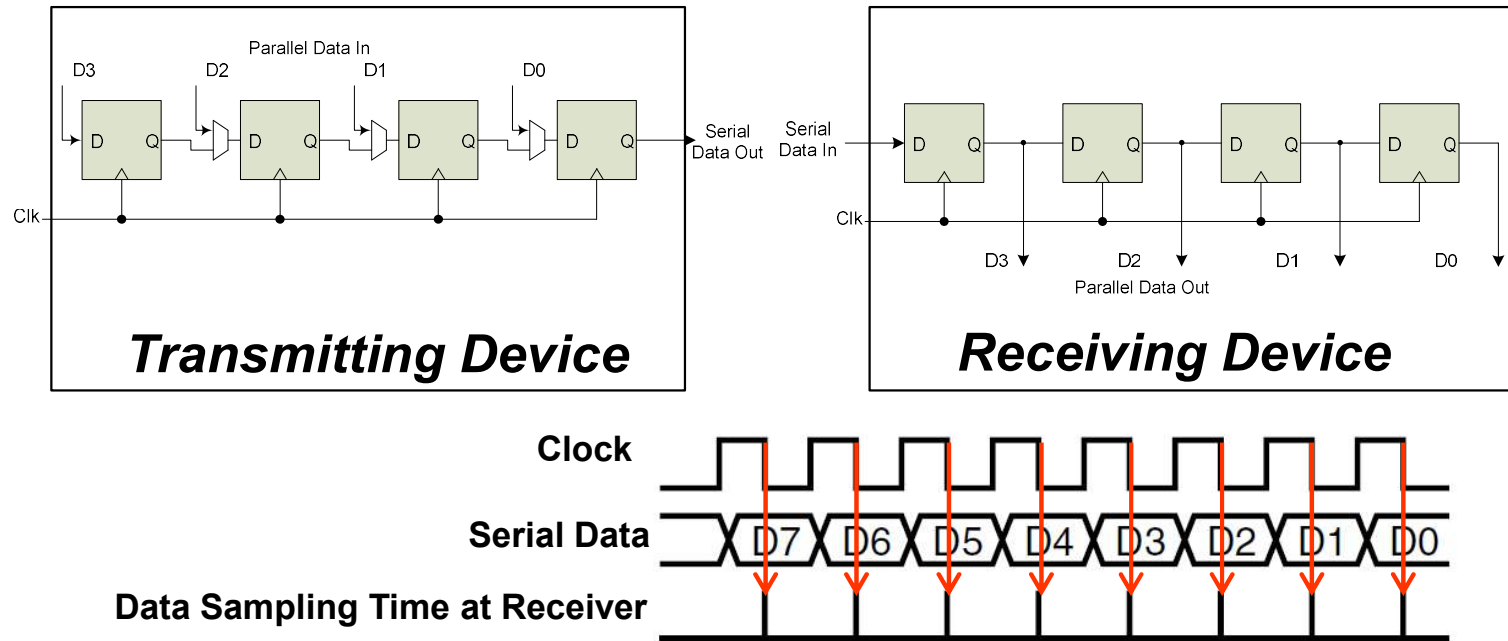
- Dedicated point-to-point connections
 - Parallel data lines, read and write lines between MCU and each peripheral
- Fast, allows simultaneous transfers
- Requires many connections, PCB area, scales badly
 - Need $4 \times (8+2) = 40$ pins on MCU to communicate!

Parallel Buses



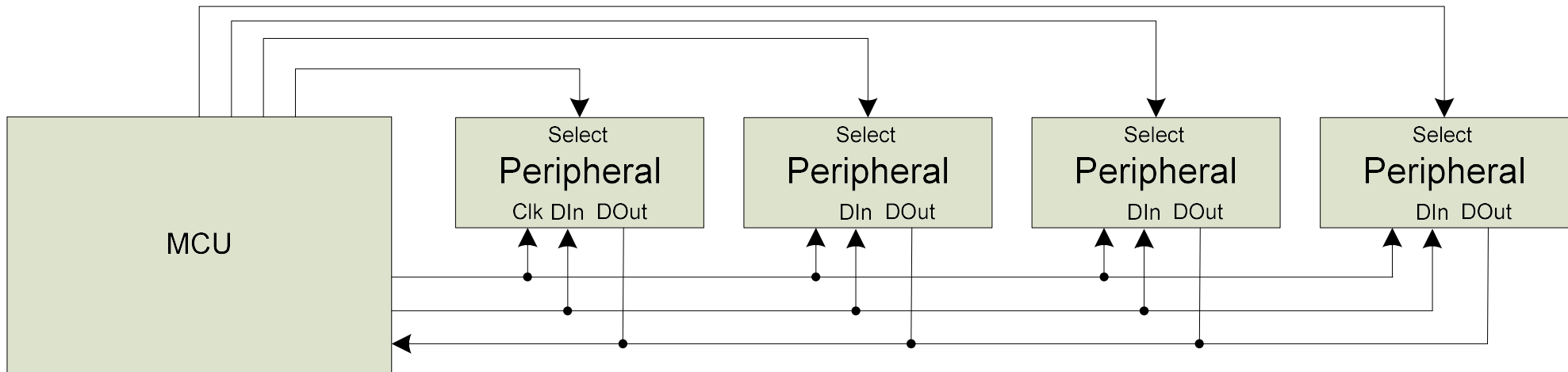
- All devices use buses to share data, read and write signals
- MCU uses individual select lines to address each peripheral
- MCU requires fewer pins for data, but still one per data bit
 - Need $4 + (8 \times 2) = 14$ pins on MCU to communicate
- MCU can communicate with only one peripheral at a time

Synchronous Serial Data Transmission



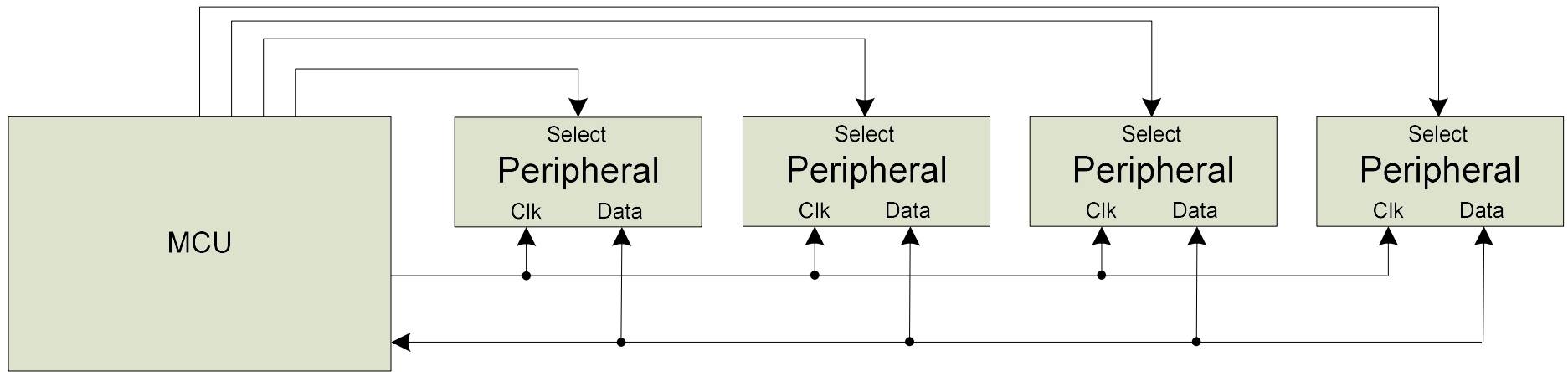
- Use shift registers and a clock signal to convert between serial and parallel formats
- Synchronous: an explicit clock signal is along with the data signal

Synchronous Full-Duplex Serial Data Bus



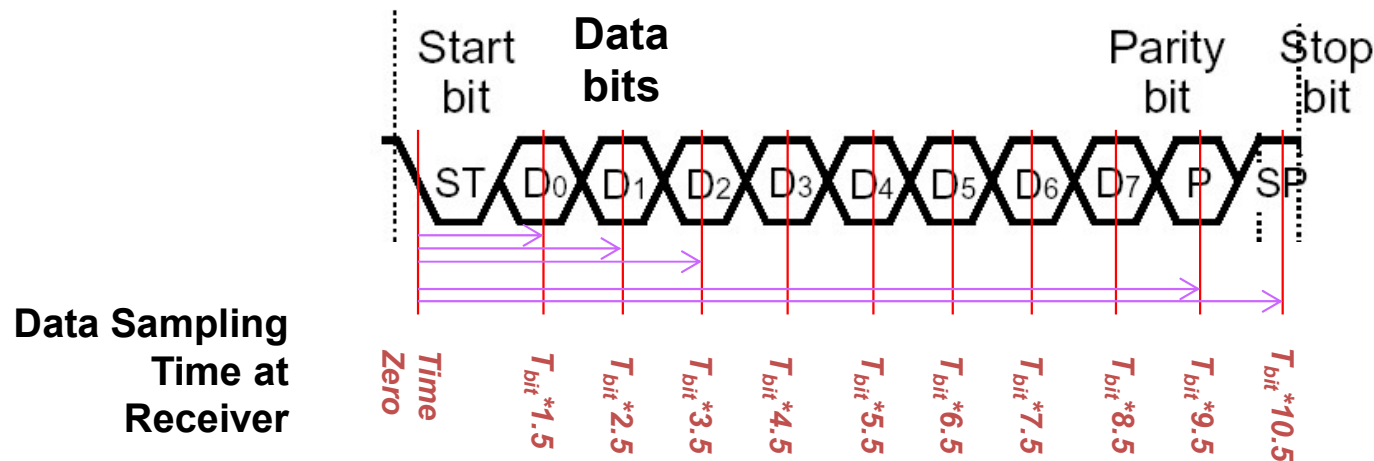
- Now can use two serial data lines - one for reading, one for writing.
 - Allows simultaneous send and receive *full-duplex communication*
 - Need $4 + 3 = 7$ pins on MCU to communicate

Synchronous Half-Duplex Serial Data Bus



- **Share the serial data line**
 - Need $4 + 2 = 6$ pins on MCU to communicate
- **Doesn't allow simultaneous send and receive - is *half-duplex communication***

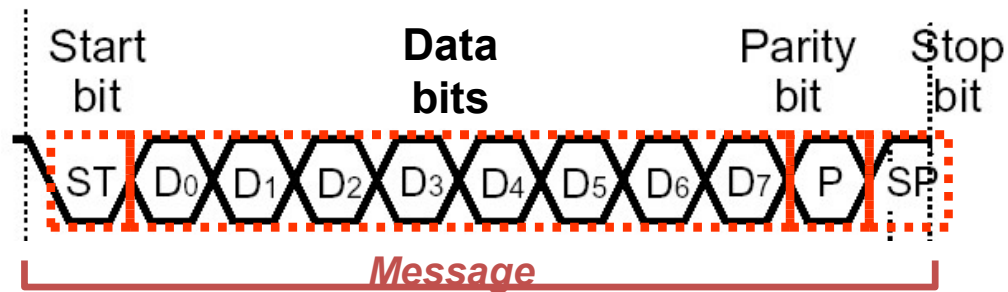
Asynchronous Serial Communication



- Eliminate the clock line!
- Transmitter and receiver must generate clock *locally*
- Transmitter must add start bit (always same value) to indicate start of each data frame
- Receiver detects leading edge of start bit, then uses it as a timing reference for sampling data line to extract each data bit N at time $T_{bit} * (N+1.5)$
- Stop bit is also used to detect some timing errors

Serial Communication Specifics

- Data frame fields
 - Start bit (one bit)
 - Data (LSB first or MSB, and size – 7, 8, 9 bits)
 - Optional parity bit is used to make total number of ones in data even or odd
 - Stop bit (one or two bits)
- All devices must use the same communications parameters
 - E.g. communication speed (300 baud, 600, 1200, 2400, 9600, 14400, 19200, etc.)
- Sophisticated network protocols have more information in each data frame
 - Medium access control – when multiple nodes are on bus, they must arbitrate for permission to transmit
 - Addressing information – for which node is this message intended?
 - Larger data payload
 - Stronger error detection or error correction information
 - Request for immediate response (“in-frame”)



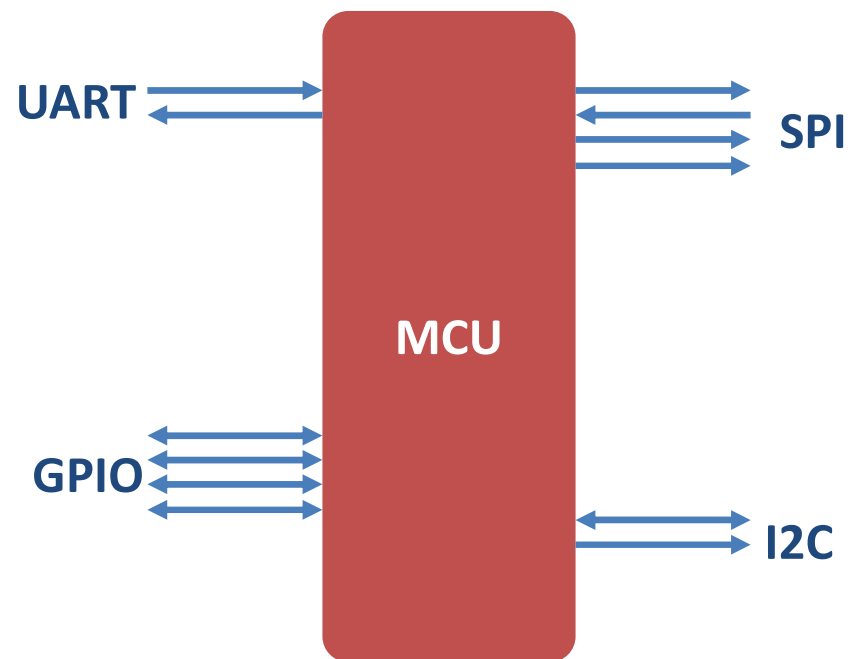
Error Detection

- Can send additional information to verify data was received correctly
- Need to specify which parity to expect: even, odd or none.
- Parity bit is set so that total number of “1” bits in data and parity is even (for even parity) or odd (for odd parity)
 - 01110111 has 6 “1” bits, so parity bit will be 1 for odd parity, 0 for even parity
 - 01100111 has 5 “1” bits, so parity bit will be 0 for odd parity, 1 for even parity
- Single parity bit detects if 1, 3, 5, 7 or 9 bits are corrupted, but doesn’t detect an even number of corrupted bits
- Stronger error detection codes (e.g. Cyclic Redundancy Check) exist and use multiple bits (e.g. 8, 16), and can detect many more corruptions.
 - Used for CAN, USB, Ethernet, Bluetooth, etc.

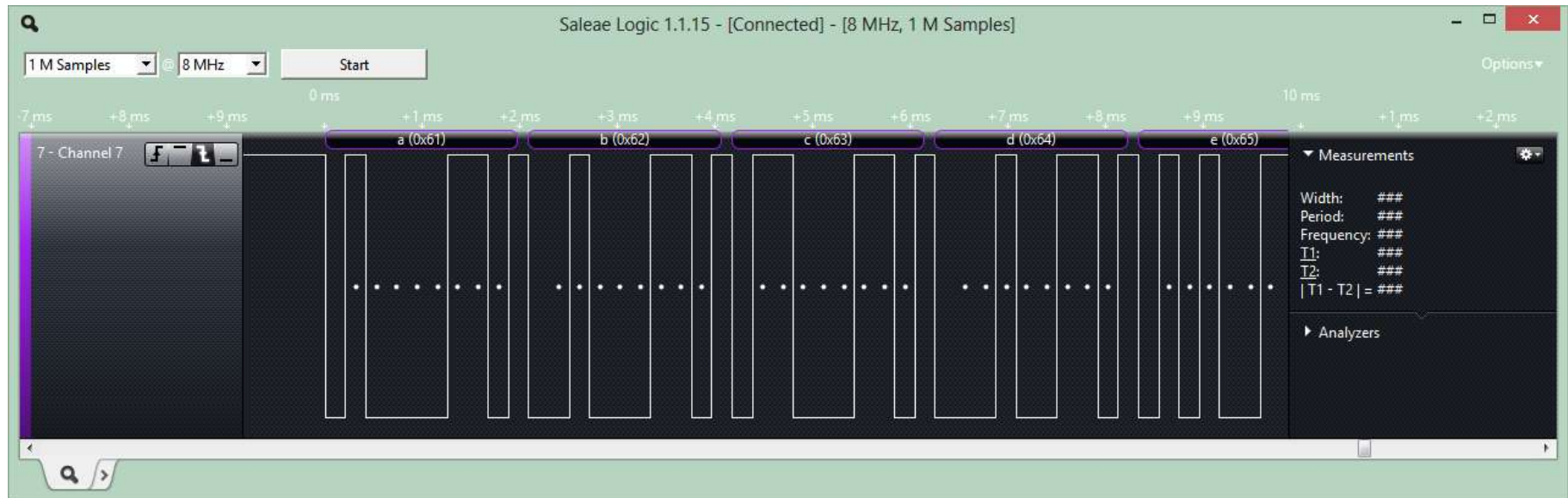
Input/Output

I/O defines how an MCU can interact with the environment

- Different I/O protocols are available
- Universal Asynchronous Receiver/Transmitter (UART) – two wires to send/receive data between devices
- Serial Peripheral Interface (SPI) – any number of bits can be sent/received without interruption
- Inter-Integrated Circuit (I²C) – combines features of UART and SPI
- General purpose I/O (GPIO) – controllable by user at run time.



Tools for Serial Communications Development



- Tedious and slow to debug serial protocols with just an oscilloscope
- Instead use a logic analyzer to decode bus traffic
- Worth its weight in gold!
- Saelae 8-Channel Logic Analyzer
 - \$150 (www.saelae.com)
 - Plugs into PC's USB port
 - Decodes SPI, asynchronous serial, I²C, 1-Wire, CAN, etc.
- Build your own: with Logic Sniffer or related open-source project

Outline



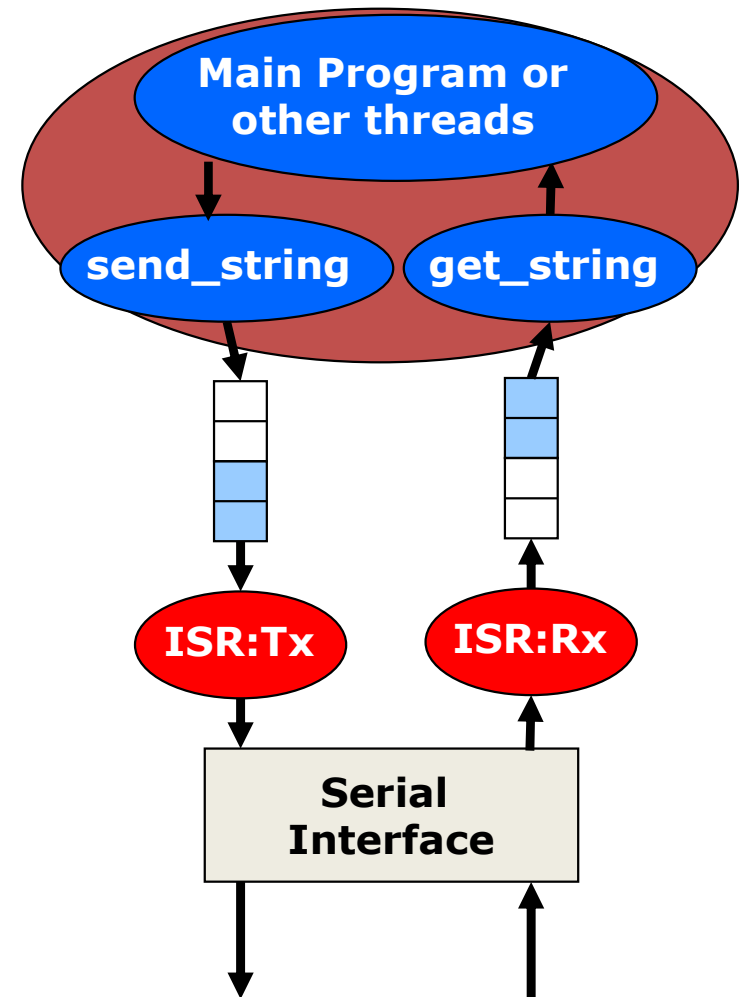
- Serial communication Concept
- **Software Architecture for Handling asynchronous Communication**
- SPI Communications
- I2C Communications
- Asynchronous serial (UART) Communications
- Protocol Comparison
- Summary

Software Structure

- Communication is *asynchronous* to program
 - Don't know what code the program will be executing ...
 - when the next item arrives
 - when current outgoing item completes transmission
 - when an error occurs
 - Need to synchronize between program and serial communication interface somehow
- Options
 - Polling
 - Wait until data is available
 - Simple but inefficient of processor time
 - Interrupt
 - CPU interrupts program when data is available
 - Efficient, but more complex

Serial Communications and Interrupts

- Want to provide *multiple* threads of control in the program
 - Main program (and subroutines it calls)
 - ISR(s)
 - Transmit ISR activity – executes when serial interface is ready to send another character
 - Receive ISR activity – executes when serial interface receives a character
 - Error ISR(s) activity – execute if an error occurs
- Need a way of buffering information between threads
 - Solution: circular queue with head and tail pointers
 - One for tx, one for rx



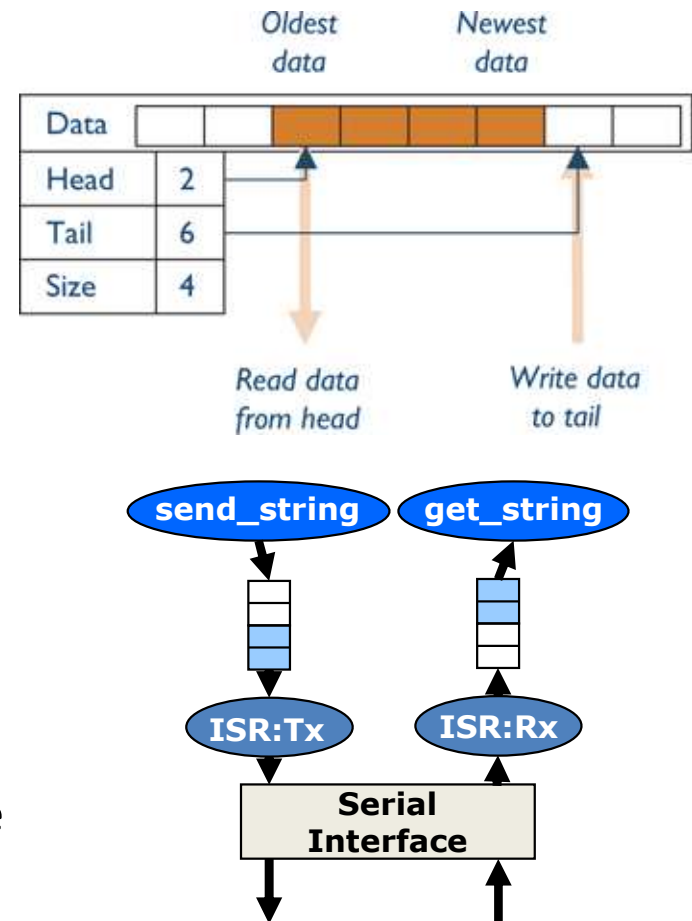
Enabling and Connecting Interrupts to ISRs

- ARM Cortex-M0+ provides one IRQ for all of a communication interface's events
- Within ISR (IRQ Handler), need to determine what triggered the interrupt, and then service it

```
void UART2_IRQHandler() {  
    if (transmitter ready) {  
        if (more data to send) {  
            get next byte  
            send it out transmitter  
        }  
    }  
    if (received data) {  
        get byte from receiver  
        save it  
    }  
    if (error occurred) {  
        handle error  
    }  
}
```

Code to Implement Queues

- Enqueue at tail: tail_ptr points to next free entry
- Dequeue from head: head_ptr points to item to remove
- #define the queue size to make it easy to change
- One queue per direction
 - ISR unloads tx_q for transmit
 - ISR loads rx_q for receive
- Other threads (e.g. main) load tx_q and unload rx_q
- Need to wrap pointer at end of buffer to make it circular,
 - Use % (modulus, remainder) operator if queue size is not power of two
 - Use & (bitwise and) if queue size is a power of two
- Queue is empty if size == 0
- Queue is full if size == Q_SIZE



Defining the Queues

```
#define Q_SIZE (32)

typedef struct {
    unsigned char Data[Q_SIZE];
    unsigned int Head; // points to oldest data element
    unsigned int Tail; // points to next free space
    unsigned int Size; // quantity of elements in queue
} volatile Q_T;

Q_T tx_q, rx_q;
```

Initialization and Status Inquiries

```
void Q_Init(Q_T * q) {  
    unsigned int i;  
    for (i=0; i<Q_SIZE; i++)  
        q->Data[i] = 0; // to simplify our lives when debugging  
    q->Head = 0;  
    q->Tail = 0;  
    q->Size = 0;  
}
```

```
int Q_Empty(Q_T * q) {  
    return q->Size == 0;  
}
```

```
int Q_Full(Q_T * q) {  
    return q->Size == Q_SIZE;  
}
```

Enqueue and Dequeue

```
int Q_Enqueue(Q_T * q, unsigned char d) {
    // What if queue is full?
    if (!Q_Full(q)) {
        q->Data[q->Tail++] = d;
        q->Tail %= Q_SIZE;
        q->Size++;
        return 1; // success
    } else
        return 0; // failure
}

unsigned char Q_Dequeue(Q_T * q) {
    // Must check to see if queue is empty before dequeuing
    unsigned char t=0;
    if (!Q_Empty(q)) {
        t = q->Data[q->Head];
        q->Data[q->Head++] = 0; // to simplify debugging
        q->Head %= Q_SIZE;
        q->Size--;
    }
    return t;
}
```

Using the Queues

- Sending data:

```
if (!Queue_Full(...)) {  
    Queue_Enqueue(..., c)  
}
```

- Receiving data:

```
if (!Queue_Empty(...)) {  
    c=Queue_Dequeue(...)  
}
```

Outline

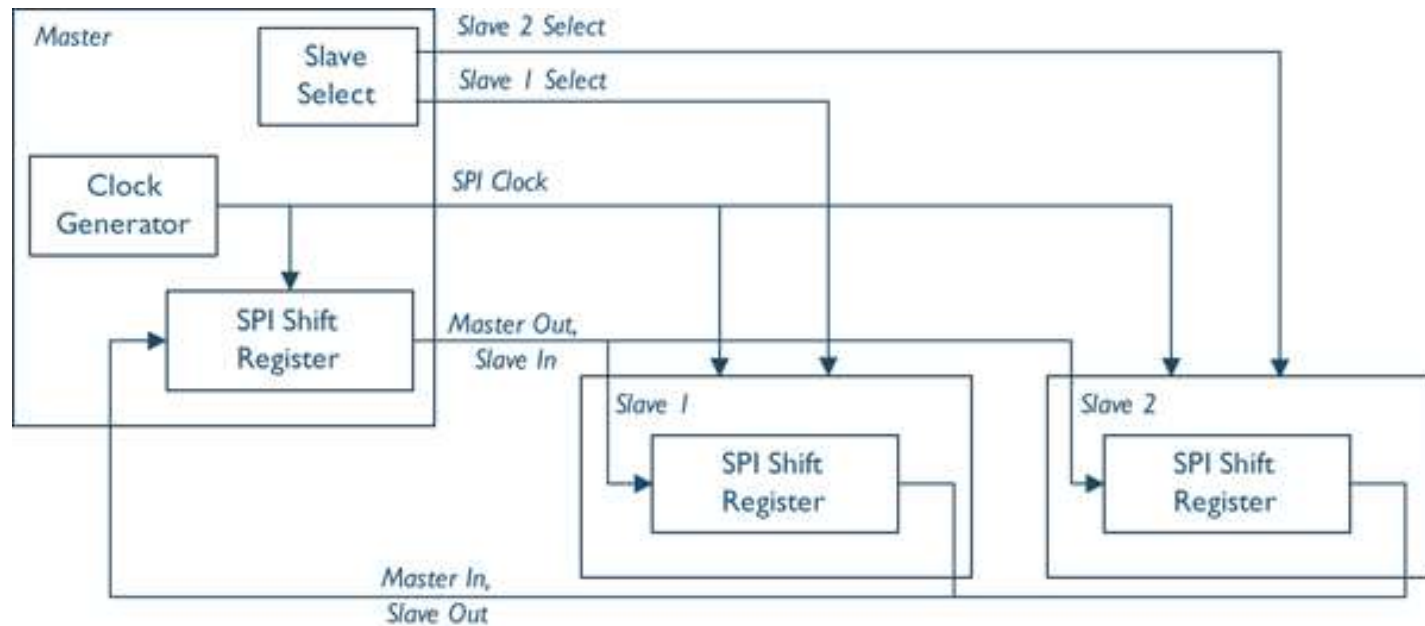
- Serial communication Concept
- Software Architecture for Handling asynchronous Communication



- **SPI Communications**

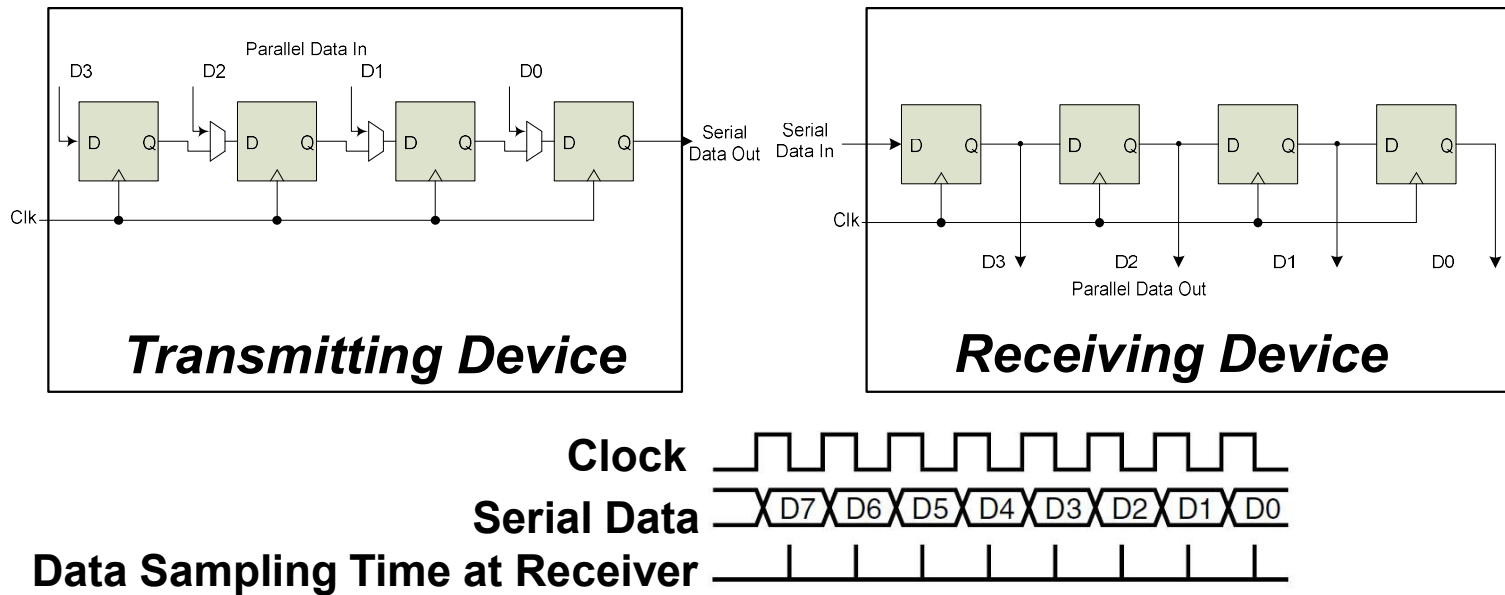
- I2C Communications
- Asynchronous serial (UART) Communications
- Protocol Comparison
- Summary

Hardware Architecture



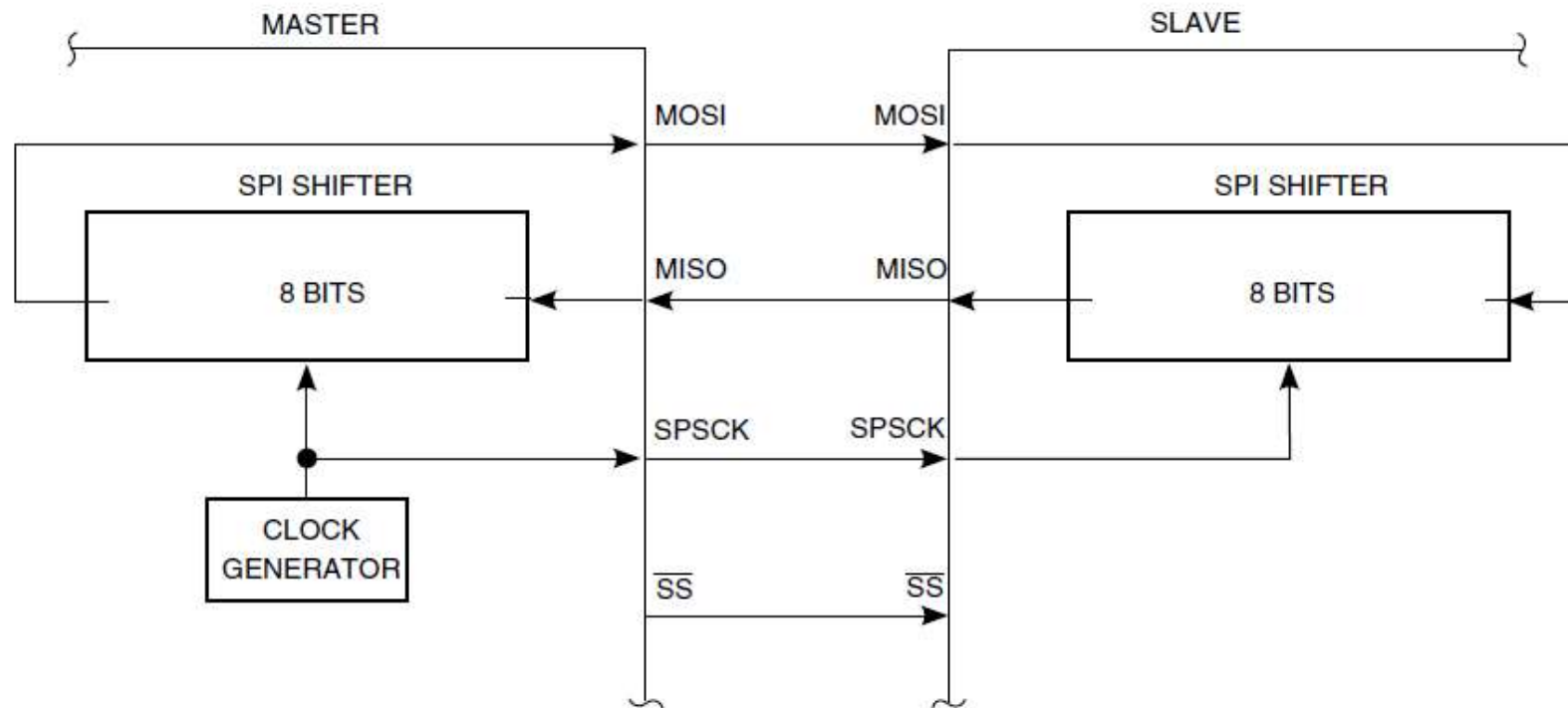
- All chips share bus signals
 - Clock SCK
 - Data lines MOSI (master out, slave in) and MISO (master in, slave out)
- Each peripheral has its own chip select line (CS)
 - Master (MCU) asserts the CS line of only the peripheral it's communicating with

Serial Data Transmission



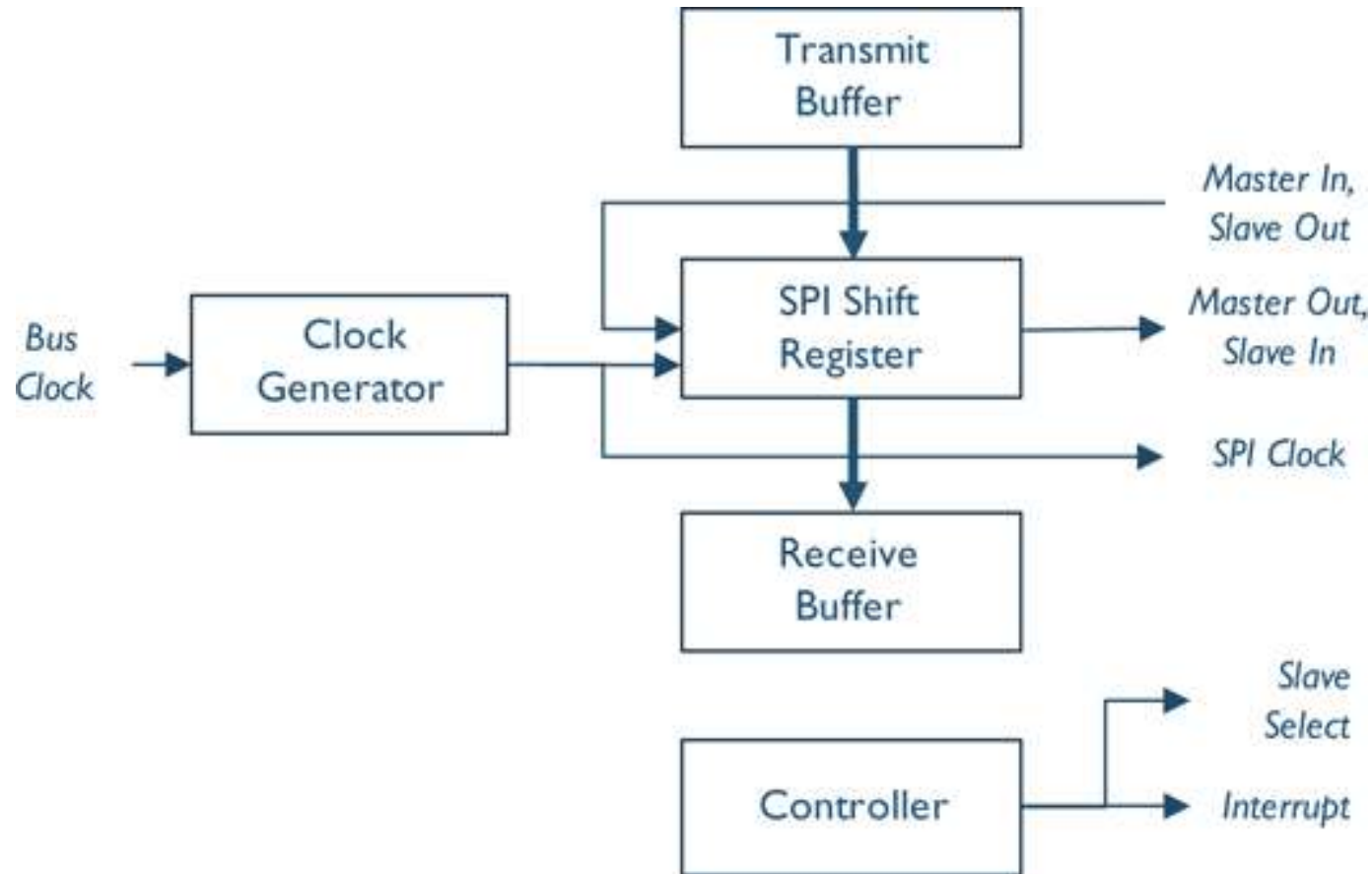
- Use shift registers and a clock signal to convert between serial and parallel formats
- Synchronous: an explicit clock signal is along with the data signal

SPI Signal Connection Overview



- SPI Communication consists of a series of data swaps between the Master and the Slave
 - As the master shifts out its transmit byte, it is also shifting in the received byte from the Slave

SPI Signal Connection Overview



- KL46Z microcontroller has two identical SPI peripherals, called SPI0 and SPI1
- Refer to Chapter 37 in the reference manual for further details

KL46Z Clock Gating for Serial Comm.

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	1				0				SPI1	SPI0	0	CMP	USBOTG	0		
W																
Reset	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0		0	UART2	UART1	UART0	0		I2C1	I2C0	1		0			
W																
Reset	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0

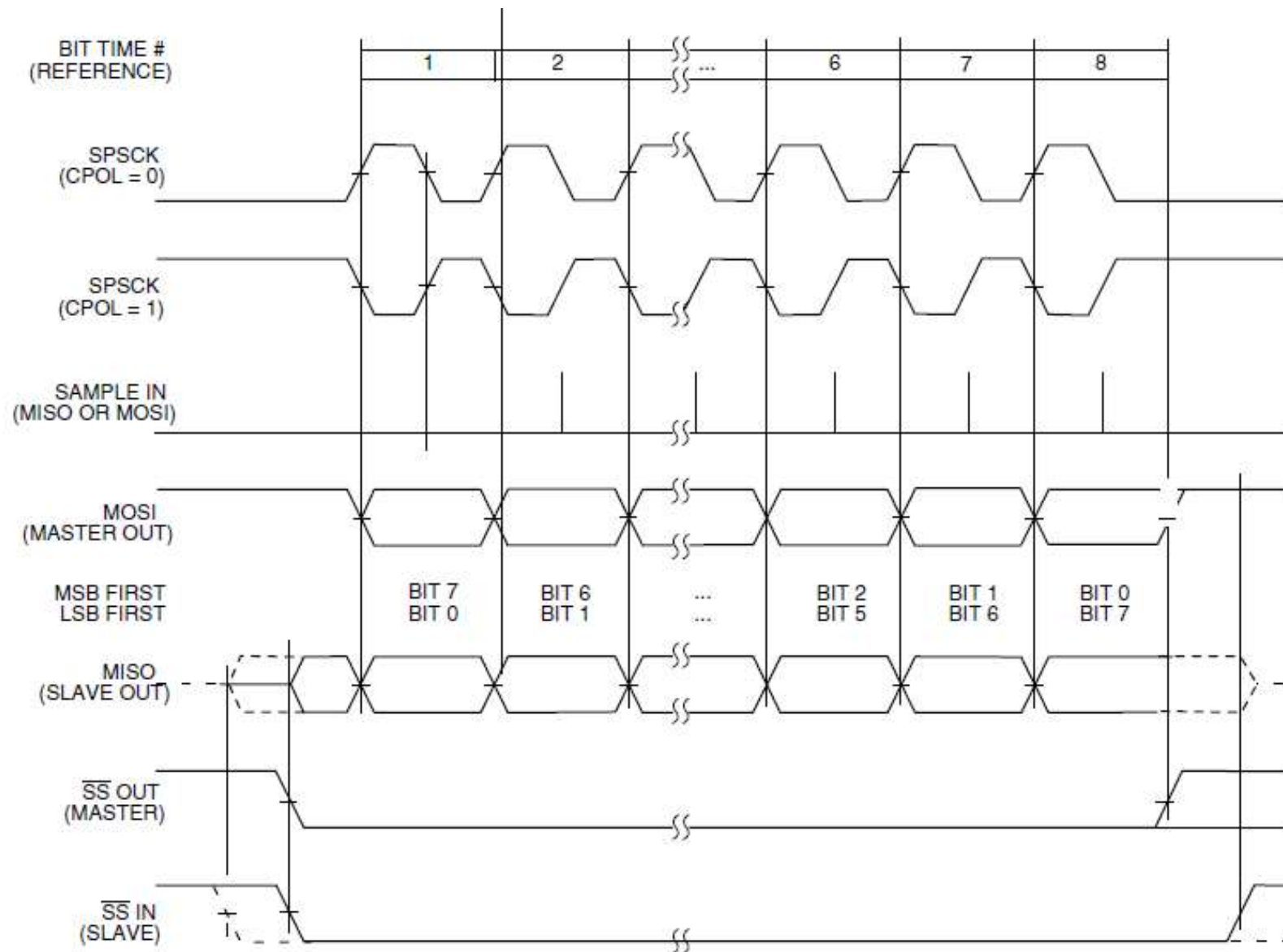
- Set corresponding bit(s) in SIM_SCGC4 Register

SPI Control Register 1 (SPIx_C1)

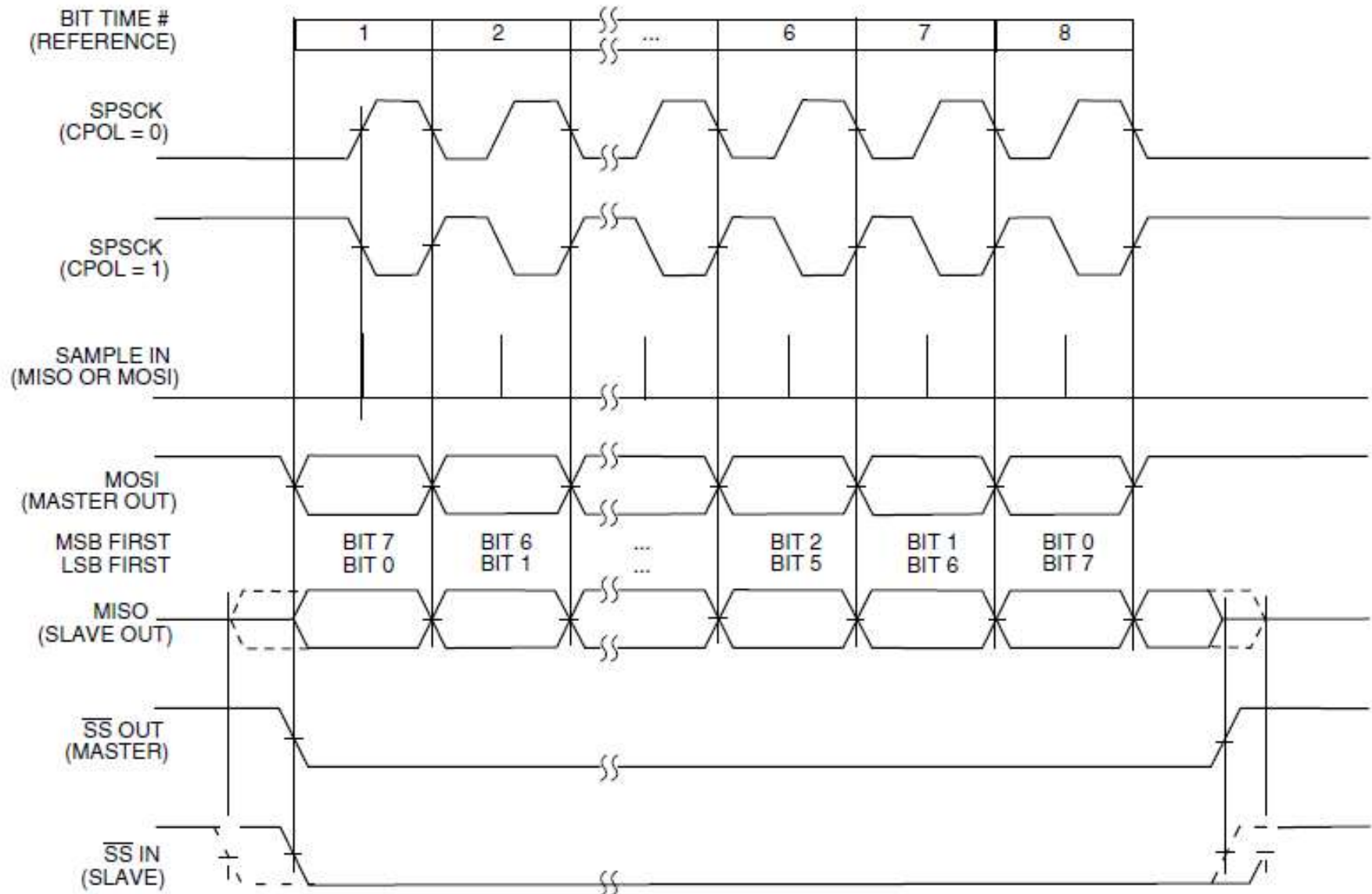
Bit	7	6	5	4	3	2	1	0
Read	SPIE	SPE	SPTIE	MSTR	CPOL	CPHA	SSOE	LSBFE
Write								
Reset	0	0	0	0	0	1	0	0

- SPIE: SPI interrupt enable for receive buffer full and mode fault
- SPE: SPI system enable
- SPTIE: SPI interrupt enable for transmit buffer empty
- MSTR: select master mode
- CPOL: Clock polarity
- CPHA: Clock phase
- SSOE: Slave select output enable
- LSBFE controls which data bit is transmitted first: the LSB (one) or the MSB (zero).

Clock and Phase Settings: CPHA = 1



Clock and Phase Settings: CPHA = 0

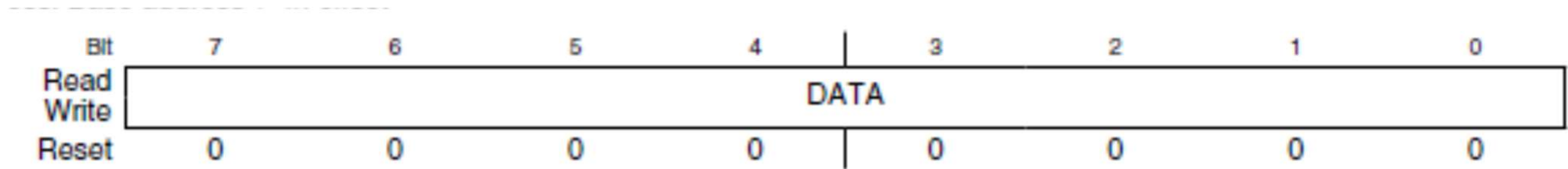


SPI Control Register 2 (SPIx_C2)

Bit	7	6	5	4	3	2	1	0
Read	SPMIE	SPLPIE	TXDMAE	MODFEN	BIDIROE	RXDMAE	SPISWAI	SPC0
Write								
Reset	0	0	0	0	0	0	0	0

- SPMIE: SPI interrupt enable for receive data match
- SPLPIE: SPI interrupt enable for wake from low-power mode
- TXDMAE: Transmit DMA enable
- MODFEN: Master mode-fault function enable
- BIDIROE: bidirectional output enable
- RDDMAE: Receive DMA enable
- SPISWAI: Stop SPI in wait mode
- SPC0: Single wire (bidirectional) mode

SPI Data Register – SPIx_D



- 8-bit data register
- A write-to SPIx_D writes to the transmit buffer
- A Read of SPIx_D reads from the receive buffer

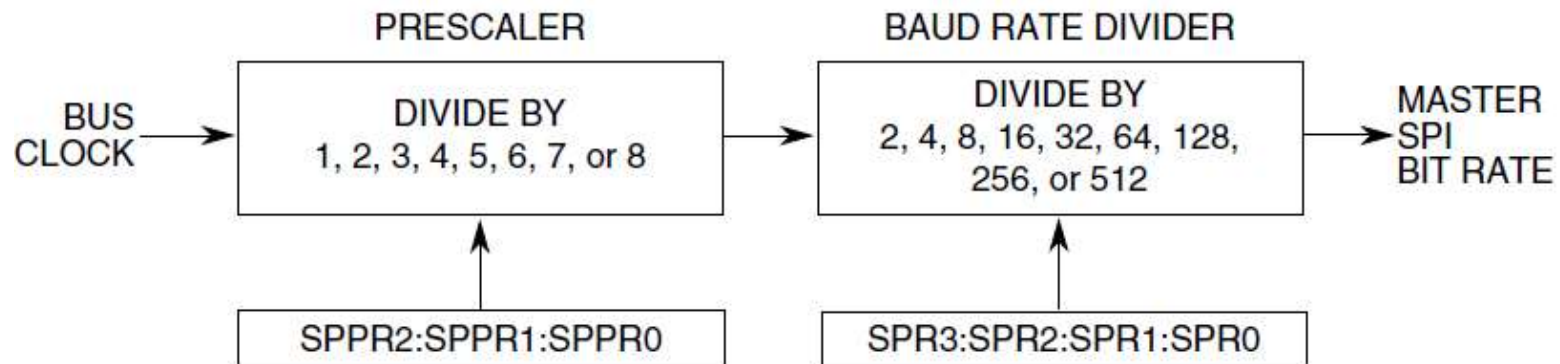
Normal and Bidirectional Modes

When SPE = 1	Master Mode MSTR = 1	Slave Mode MSTR = 0
Normal Mode SPC0 = 0		
Bidirectional Mode SPC0 = 1		

SPI Baud Rate Register (SPIx_BR)

Bit	7	6	5	4	3	2	1	0
Read	0	SPPR[2:0]			SPR[3:0]			
Write								
Reset	0	0	0	0	0	0	0	0

- SPPR: SPI baud rate prescale divisor: divides by n+1
- SPR: SPI baud rate divisor: divides by 2^{n+1}



- $$f_{\text{SPI}} = f_{\text{bus_clock}} / ((\text{SPPR} + 1) * 2^{\text{SPR} + 1})$$

SPI Status Register (SPIx_S)

7	6	5	4	3	2	1	0
SPRF	SPMF	SPTEF	MODF	0			

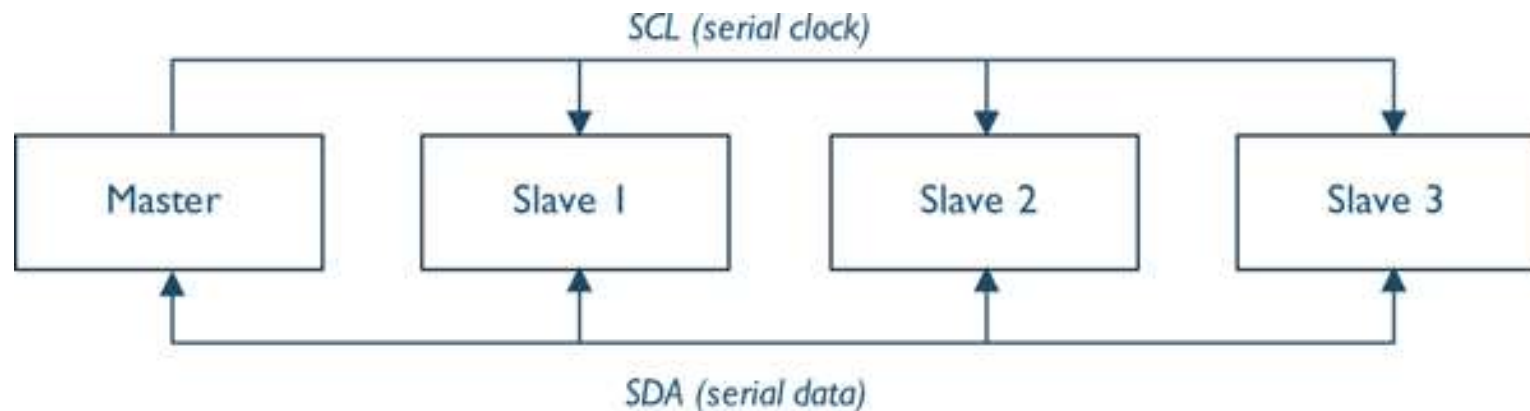
- The SPTEF bit will be set if the SPIx transmit buffer is empty, indicating the transmit buffer can be loaded with new data.
- The SPRF bit will be set at the end of an SPI transfer when the SPIx receive buffer is full.
- The SPMF bit indicates that received data matches the contents of the SPIx_M (match) register.
- The MODF bit indicates a mode fault, in which multiple masters attempt to drive the SPI clock and MOSI signals.

Outline

- Serial communication Concept
- Software Architecture for Handling asynchronous Communication
- SPI Communications
- **I2C Communications**
- Asynchronous serial (UART) Communications
- Protocol Comparison
- Summary

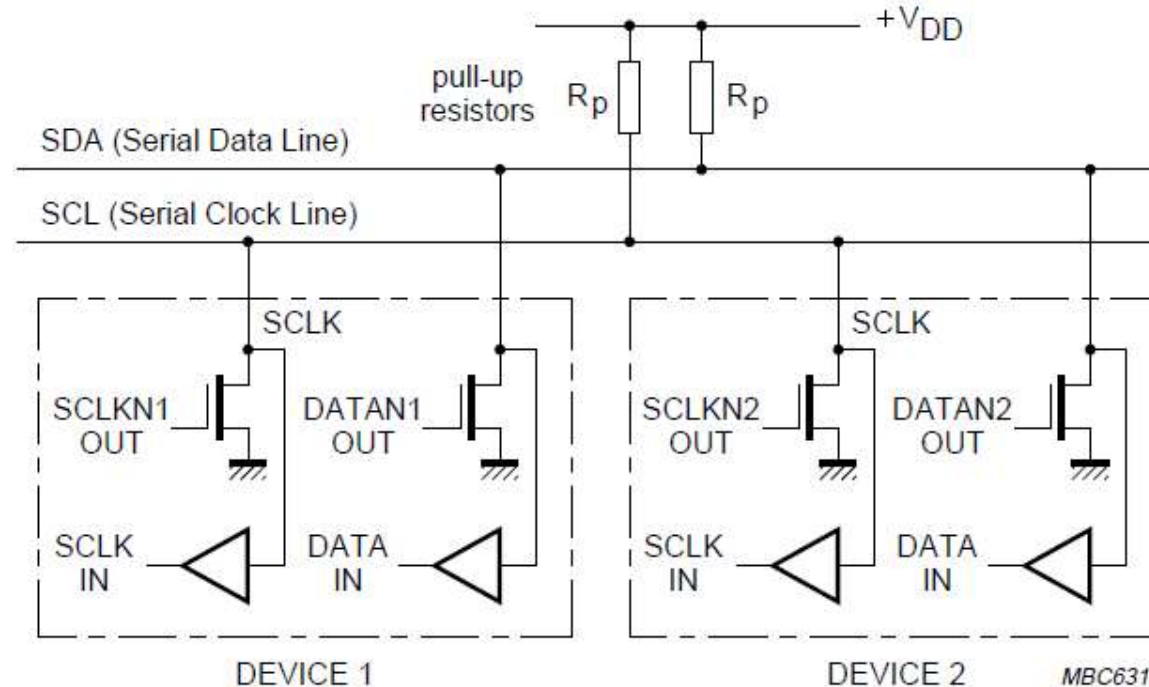


I2C Bus Overview



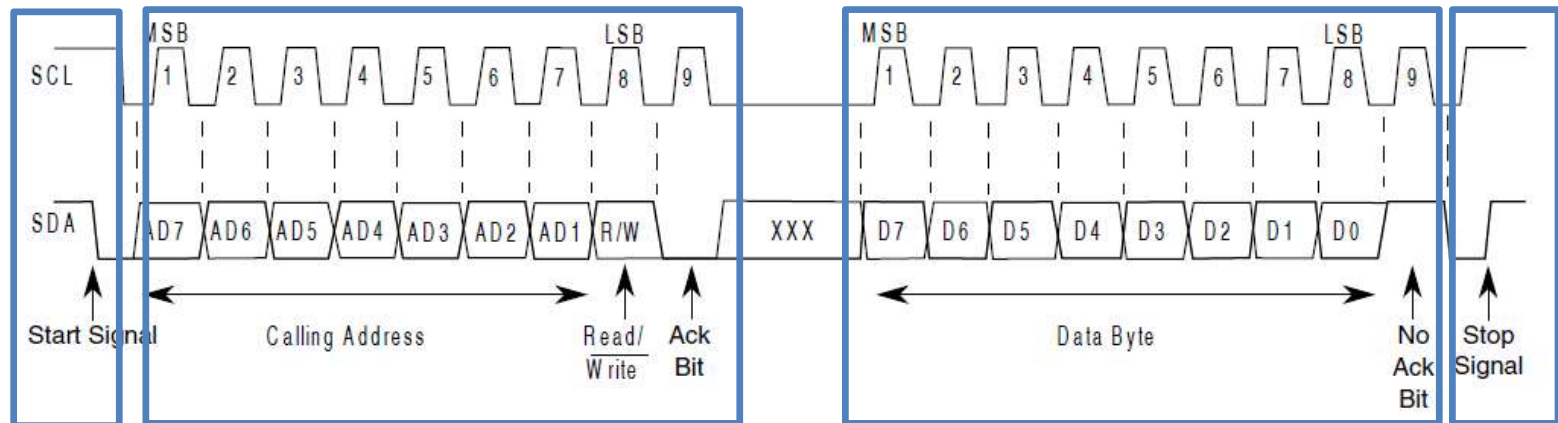
- “Inter-Integrated Circuit” bus
- Multiple devices connected by a shared serial bus
- Bus is typically controlled by master device, slaves respond when addressed
- I²C bus has two signal lines
 - SCL: Serial clock
 - SDA: Serial data
- Full details available in “The I²C-bus Specification”

I2C Bus Connections



- Resistors pull up lines to V_{DD}
- Open-drain transistors pull lines down to ground
- Master generates SCL clock signal
 - Can range up to 400 kHz, 1 MHz, or more

I2C Message Format



- **Message-oriented data transfer with four parts**

1. Start condition
2. Slave Address transmission
 - Address
 - Command (read or write)
 - Acknowledgement by receiver
3. Data fields
 - Data byte

- Acknowledgement by receiver
- 4. Stop condition

- **Message is made of**

- Signals: Start, Stop, Repeated Start
- Bytes
- Acknowledgement bits

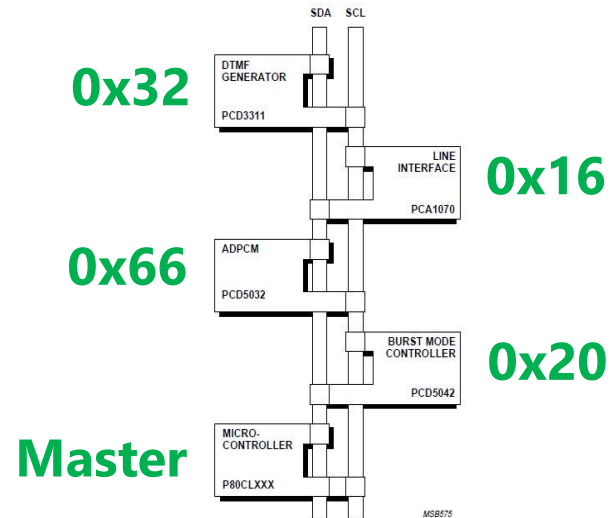
Device Addressing

	Slave Device Address									Data Byte									
Start	AD7	AD6	AD5	AD4	AD3	AD2	AD1	R/W	ACK	D7	D6	D5	D4	D3	D2	D1	D0	ACK	Stop

- The master uses the device address to select a particular slave device on the bus
- The first byte in the message has two parts
 - Device address: the upper seven bits,
 - R/W bit: the LSB bit; 1 – read, 0 - write

I2C Addressing: Devices and Registers

- Slave device addressing
 - Each slave device has a seven-bit address
 - Can support up to $2^7=128$ different devices on same bus
 - Different types of device have different default addresses
 - Sometimes can select a secondary default address by tying a device pin to a different logic level
- Register addressing
 - I2C devices may have multiple control, status, data registers and even data memory internally – how do we get at it?
 - Use the first byte of data as a **register** address
 - Example: First seven registers of MMA8451 I2C accelerometer



Name	Type	Register Address	Comment
STATUS/F_STATUS ⁽¹⁾⁽²⁾	R	0x00	FMODE = 0, real time status FMODE > 0, FIFO status
OUT_X_MSB ⁽¹⁾⁽²⁾	R	0x01	[7:0] are 8 MSBs of 14-bit sample. Root pointer to XYZ FIFO data.
OUT_X_LSB ⁽¹⁾⁽²⁾	R	0x02	[7:2] are 6 LSBs of 14-bit real-time sample
OUT_Y_MSB ⁽¹⁾⁽²⁾	R	0x03	[7:0] are 8 MSBs of 14-bit real-time sample
OUT_Y_LSB ⁽¹⁾⁽²⁾	R	0x04	[7:2] are 6 LSBs of 14-bit real-time sample
OUT_Z_MSB ⁽¹⁾⁽²⁾	R	0x05	[7:0] are 8 MSBs of 14-bit real-time sample
OUT_Z_LSB ⁽¹⁾⁽²⁾	R	0x06	[7:2] are 6 LSBs of 14-bit real-time sample

I2C with Register Addressing

SINGLE-BYTE WRITE									
MASTER	START	SLAVE ADDRESS + WRITE		REGISTER ADDRESS		DATA		STOP	
SLAVE			ACK		ACK		ACK		

MULTIPLE-BYTE WRITE									
MASTER	START	SLAVE ADDRESS + WRITE		REGISTER ADDRESS		DATA		DATA	STOP
SLAVE			ACK		ACK		ACK		ACK

SINGLE-BYTE READ									
MASTER	START	SLAVE ADDRESS + WRITE		REGISTER ADDRESS		START ¹	SLAVE ADDRESS + READ		
SLAVE			ACK		ACK			ACK	DATA

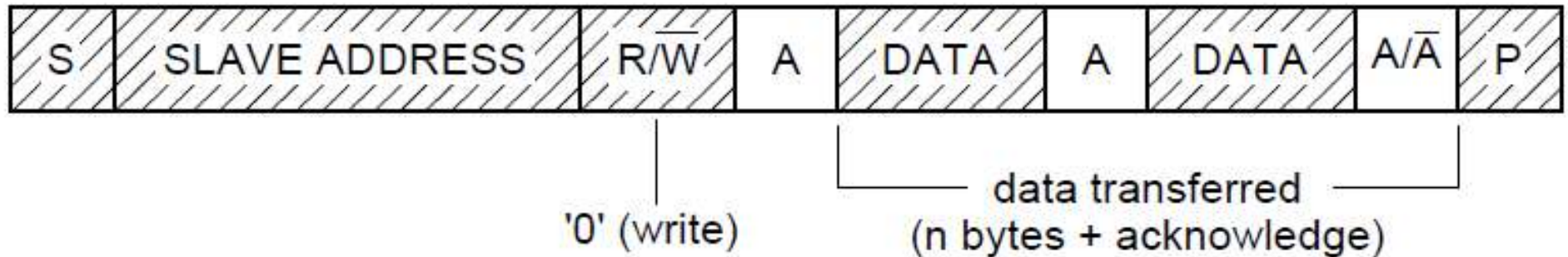
MULTIPLE-BYTE READ									
MASTER	START	SLAVE ADDRESS + WRITE		REGISTER ADDRESS		START ¹	SLAVE ADDRESS + READ		
SLAVE			ACK		ACK			ACK	DATA

NOTES

1. THIS START IS EITHER A RESTART OR A STOP FOLLOWED BY A START.
2. THE SHADED AREAS REPRESENT WHEN THE DEVICE IS LISTENING.

- Master drives communication
 - Sends start condition, address of slave, read/write command
 - Listens for acknowledgement from slave
 - Sends register address (byte)
 - Listens for acknowledgement from slave

Master Writing Data to Slave



from master to slave



from slave to master

MBC605

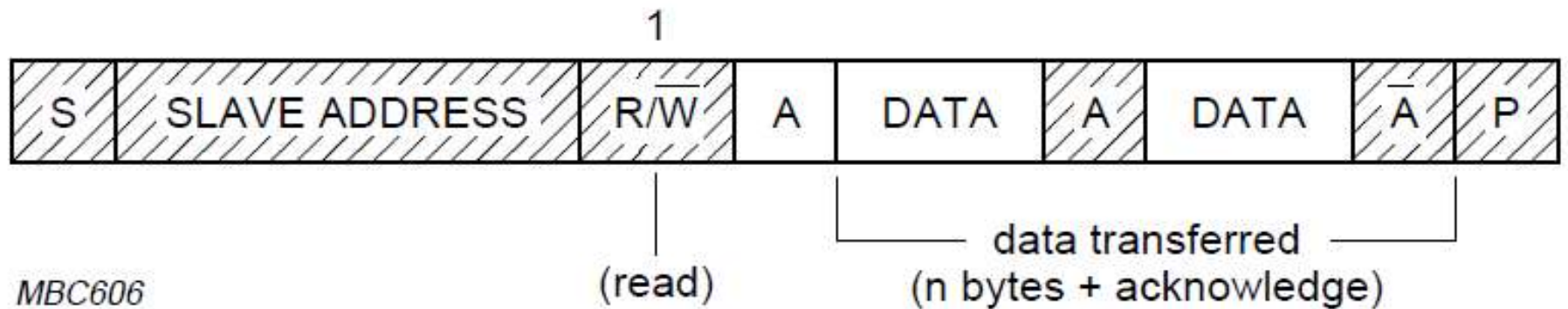
A = acknowledge (SDA LOW)

$\bar{A}$ = not acknowledge (SDA HIGH)

S = START condition

P = STOP condition

Master Reading Data from Slave



 from master to slave

 from slave to master

MBC605

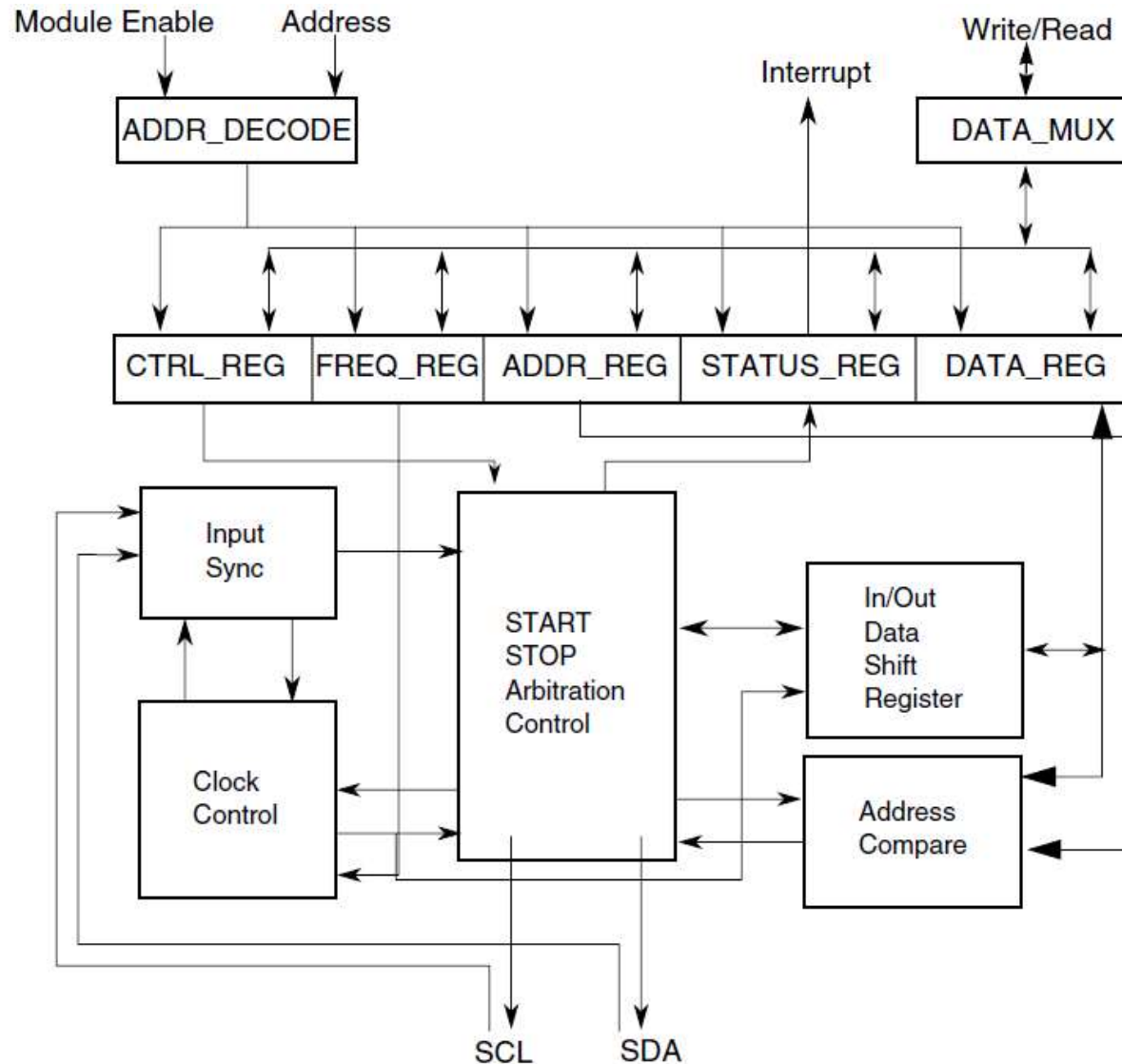
A = acknowledge (SDA LOW)

$\bar{A}$ = not acknowledge (SDA HIGH)

S = START condition

P = STOP condition

KL46Z I2C Controller



- The KL46Z microcontroller has two I2C peripherals, called I2C0 and I2C1
- Refer to Chapter 38 in the reference manual for further details

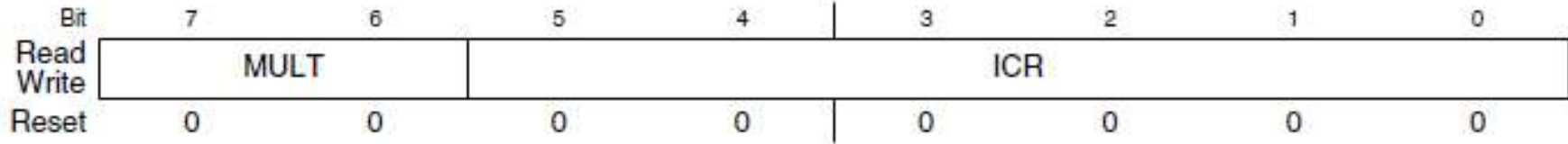
KL46Z Clock Gating for I2C module.

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	1				0						0				0	
W									SPI1	SPI0			CMP	USBOTG		
Reset	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0		0	UART2	UART1	UART0	0		I2C1	I2C0	1		0			
W				UART2	UART1	UART0										
Reset	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0

- Set corresponding bit(s) in SIM_SCGC4 Register

Setting the I2C Baud Rate



- I2Cx_F: Frequency Divider register
 - MULT: specified multiplier $mul = 2^{MULT}$
 - valid values: 1, 2, 4
 - ICR: Clock Rate indicates which value of SCL divider to use from a table of 64 alternatives see “I2C divider and hold values” in the reference manual
 - I2C baud rate

$$I^2C \text{ Baud rate} = \frac{\text{bus speed}}{mul \times SCL \text{ divider}}$$

I2C Control Register 1 – I2Cx_C1

Bit	7	6	5	4	3	2	1	0
Read	IICEN	IICIE	MST	TX	TXAK	0	WUEN	DMAEN
Write						RSTA		
Reset	0	0	0	0	0	0	0	0

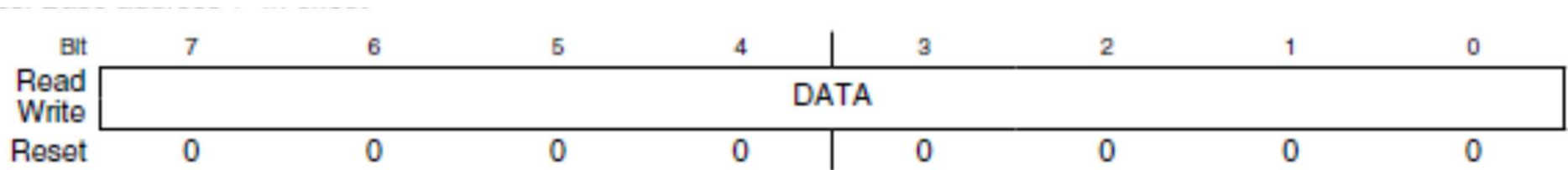
- IICEN - enable I2C module
- IICIE - enable I2C interrupt
- MST - select master mode
 - 0 → 1 generates Start condition
 - 1 → 0 generates Stop condition
- TX – selects if the peripheral will transmit (one) or receive (zero).
- TXAK – Transmit Acknowledge enable
- RSTA – Repeat Start
- WUEN – Wakeup enable
- DMAEN – Enable DMA

I2C Status Register – I2Cx_S

Bit	7	6	5	4	3	2	1	0
Read	TCF	IAAS	BUSY	ARBL	RAM	SRW	IICIF	RXAK
Write				w1c			w1c	
Reset	1	0	0	0	0	0	0	0

- **TCF – Transfer Complete flag set after transferring byte and acknowledge bit**
- IAAS – Addressed as a Slave
- **BUSY – bus busy**
- ARBL – arbitration lost
- RAM – Range address match
- SRW – when slave, indicates transmission direction (0: slave receive, 1: slave transmit)
- **IICIF: Interrupt pending flag**
- **RXAK: 0: acknowledge signal received**

I2C Data Register – I2Cx_D



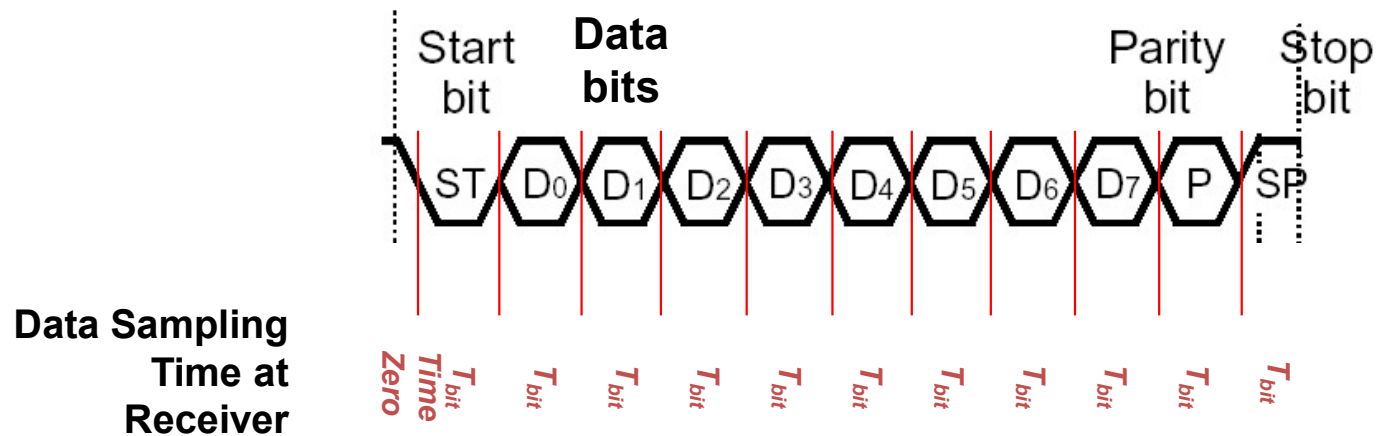
- 8-bit data register
- Master transmit mode
 - Writing to I2Cx_D starts a data transfer
- Master receive mode
 - Reading from I2Cx_D starts reception of next byte

Outline

- Serial communication Concept
- Software Architecture for Handling asynchronous Communication
- SPI Communications
- I2C Communications
- **Asynchronous serial (UART) Communications**
- Protocol Comparison
- Summary

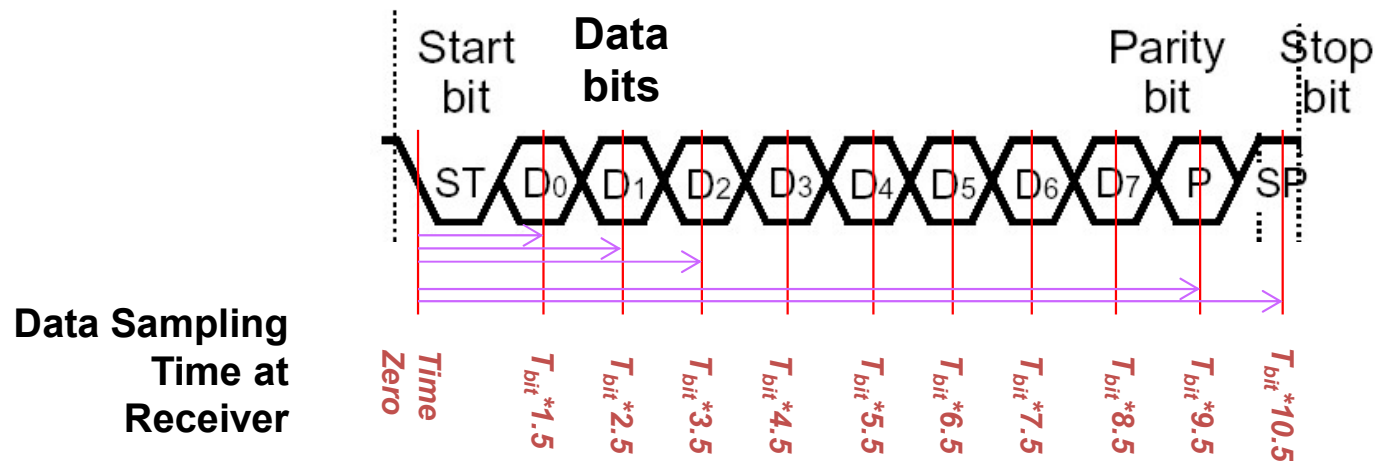


Transmitter Basics



- If no data to send, keep sending 1 (stop bit) – *idle line*
- When there is a data word to send
 - Send a 0 (start bit) to indicate the start of a word
 - Send each data bit in the word (use a shift register for the *transmit buffer*)
 - Send a 1 (stop bit) to indicate the end of the word

Receiver Basics



- Wait for a falling edge (beginning of a Start bit)
 - Then wait $\frac{1}{2}$ bit time
 - Do the following for as many data bits in the word
 - Wait 1 bit time
 - Read the data bit and shift it into a *receive buffer* (shift register)
 - Wait 1 bit time
 - Read the bit
 - if 1 (Stop bit), then OK
 - if 0, there's a problem!

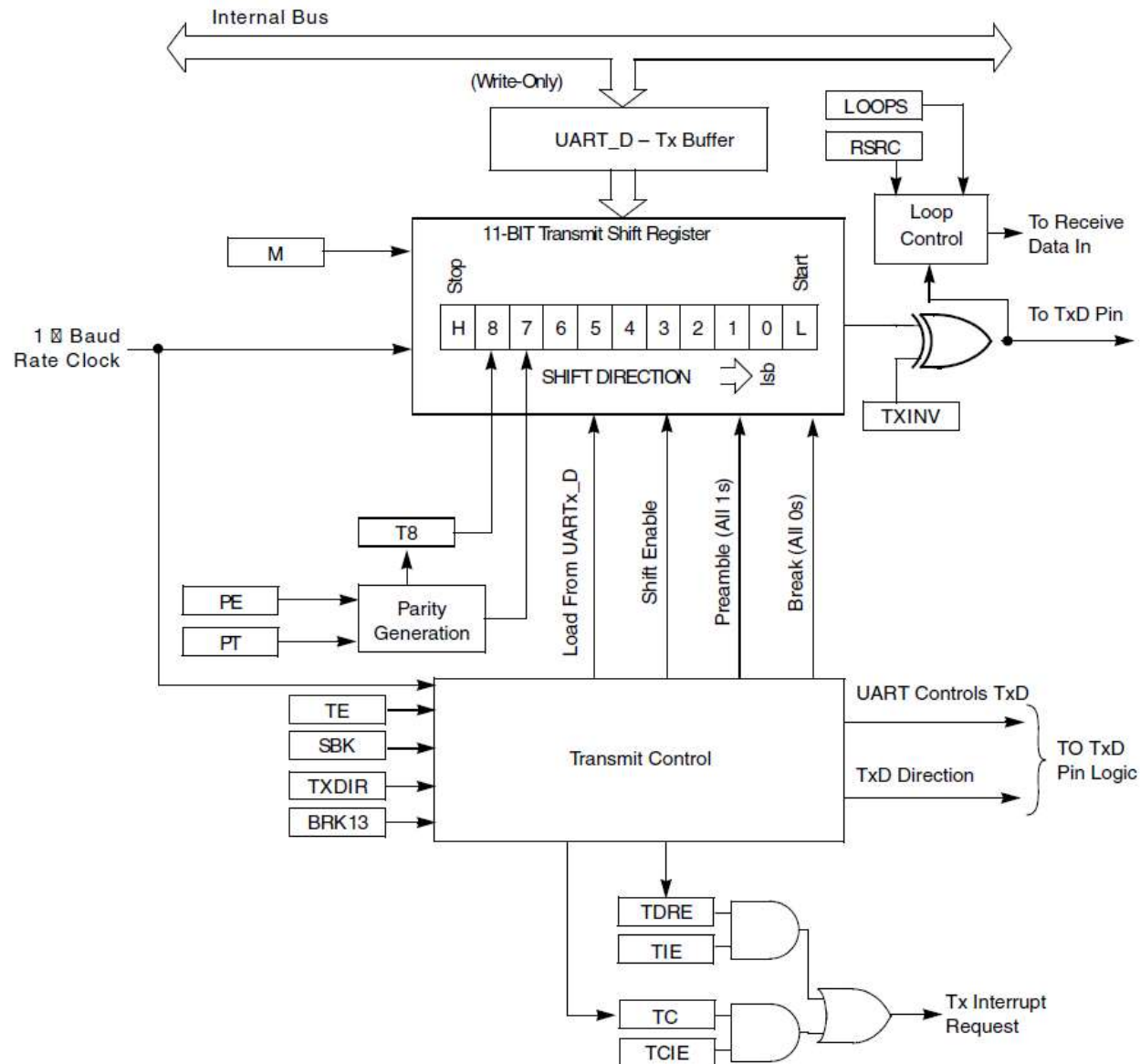
For this to work...

- Transmitter and receiver must agree on several things (protocol)
 - Order of data bits
 - Number of data bits
 - What a start bit is (1 or 0)
 - What a stop bit is (1 or 0)
 - How long a bit lasts
 - Transmitter and receiver clocks must be reasonably close in frequency, since the only timing reference is the start of the start bit

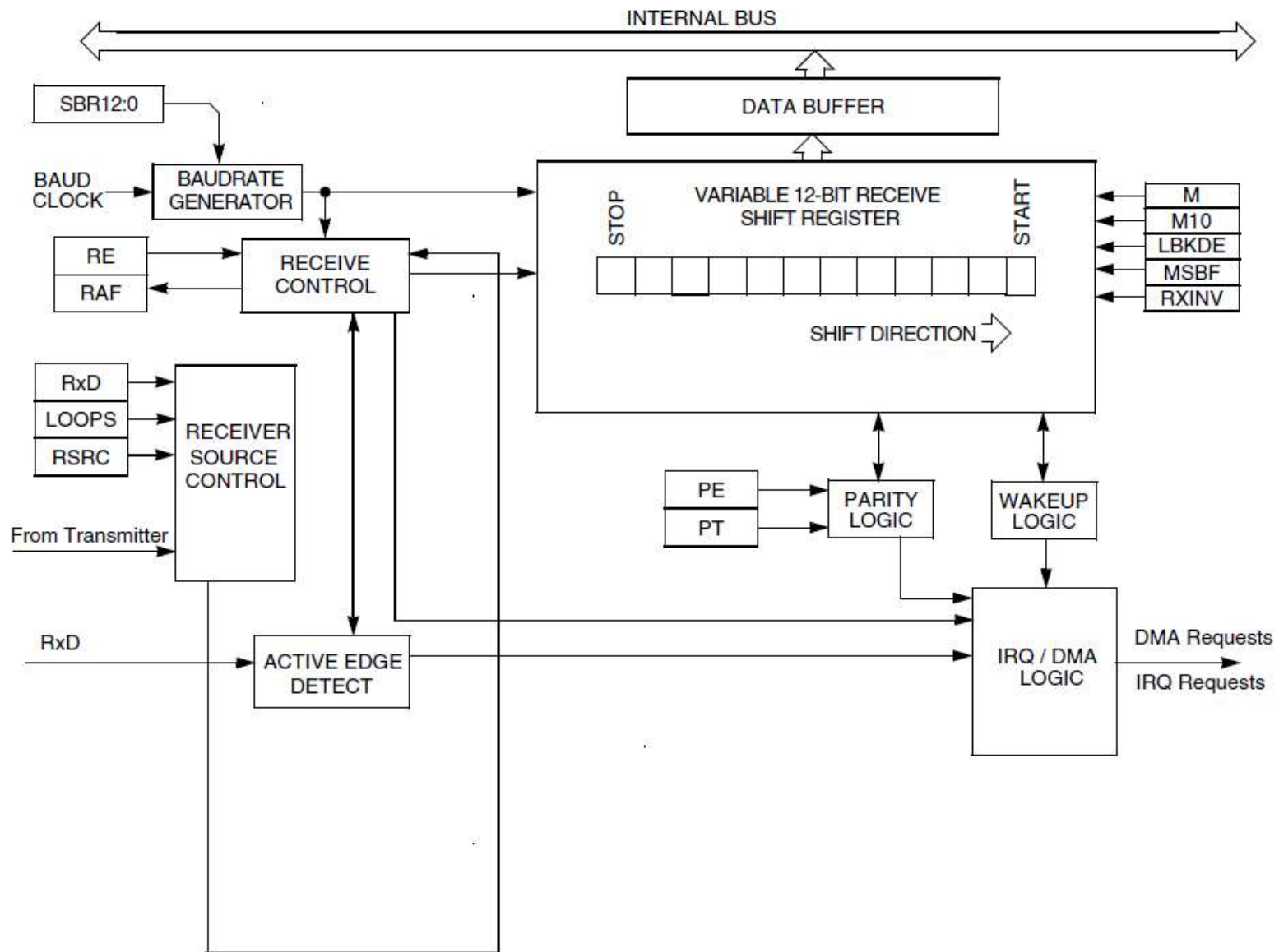
KL46 UARTs

- UART: Universal (configurable) Asynchronous Receiver/Transmitter
- UART0 (chapter 39 in the reference manual)
 - Low Power
 - Can oversample from 4x to 32x
 - Is used by debugger MCU on Freedom KL46Z, so not available
- UART1, UART2 (chapter 40 in the reference manual)
 - More basic, fewer features, easier to program

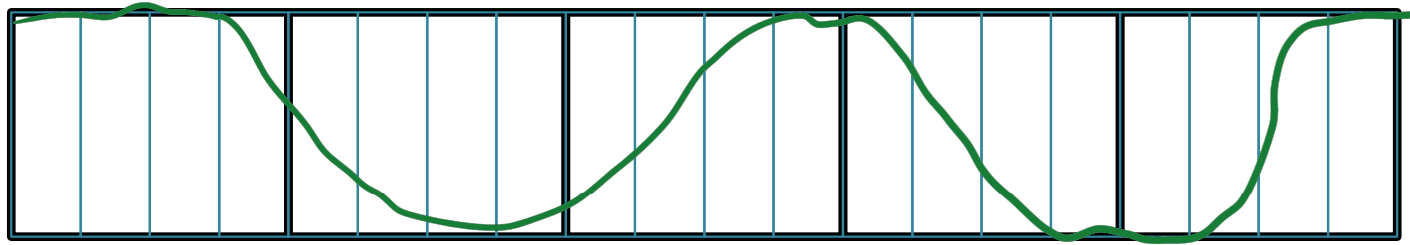
UART Transmitter



UART Receiver



Input Data Oversampling



- When receiving, UART *oversamples* incoming data line
 - Extra samples allow voting, improving noise immunity
 - Better synchronization to incoming data, improving noise immunity
- UART0 provides configurable oversampling from 4x to 32x
 - Put desired oversampling factor minus one into UART0 Control Register 4, OSR bits.
- UART1, UART2 have fixed 16x oversampling

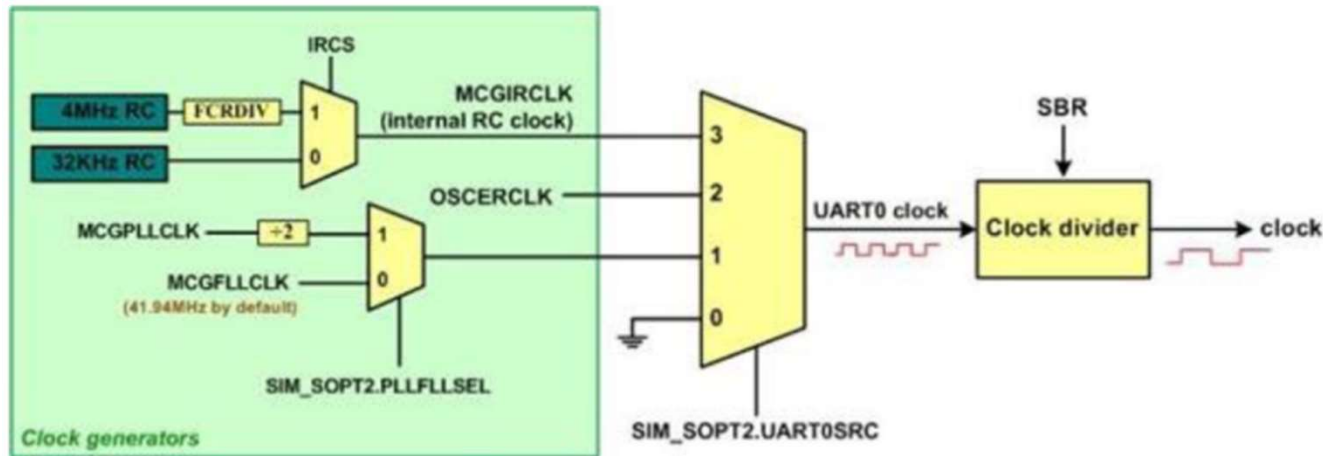
KL46Z Clock Gating for UART modules

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	1				0						0			USBOTG	0	
W									SPI1	SPI0			CMP			
Reset	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0		0	UART2	UART1	UART0	0				1		0			
W									I2C1	I2C0						
Reset	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0

- Set corresponding bit(s) in SIM_SCGC4 Register

Baud Rate Generator



- Select the source of the Baud rate generator clock for by SIM_SOPT2 register
- The transmit clock of UART0 is calculated by the clock source by the content of UART0_BDH:UART0_BDL and then by the Over Sampling Ratio (OSR):

$$\text{Baud Rate} = \frac{\text{Clock Source}}{(\text{OSR} + 1) \times \text{SBR}}$$

When SBR = UART0_BDL / (UART0_BDH + 1)

Using the UART

- When can we transmit?
 - Transmit buffer must be empty
 - Can poll UARTx->S1 TDRE flag
 - Or we can use an interrupt, in which case we will need to queue up data
- Put data to be sent into UARTx_D (UARTx->D in with CMSIS)
- When can we receive a byte?
 - Receive buffer must be full
 - Can poll UARTx->S1 RDRF flag
 - Or we can use an interrupt, and again we will need to queue the data
- Get data from UARTx_D (UARTx->D in with CMSIS)

UART Control Register 1 (UART0_C1)

Bit	7	6	5	4	3	2	1	0
Read	LOOPS	DOZEEN	RSRC	M	WAKE	ILT	PE	PT
Write								
Reset	0	0	0	0	0	0	0	0

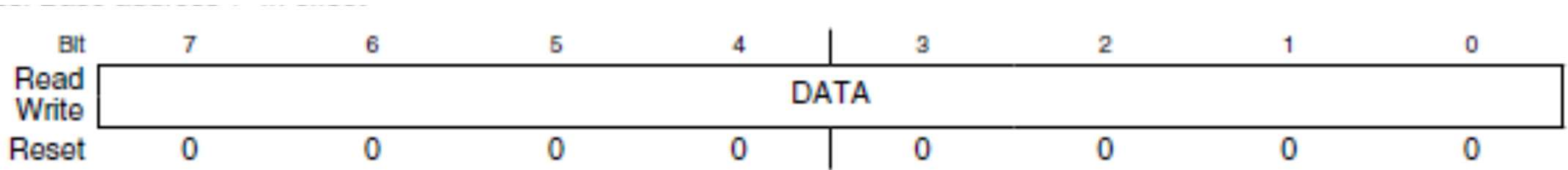
- LOOPS: Enables loopback/single-pin (TX/RX) mode
- DOZEEN: Doze enable – disable UART in sleep mode
- RSRC: Selects between loopback and single-pin mode
- **M: Select 9-bit data mode (instead of 8-bit data)**
- WAKE: Wakeup method
- ILT: Idle line type
- **PE: Parity enabled with 1**
- **PT: Odd parity with 1, even parity with 0**

UART Control Register 2 (UART0_C2)

Bit	7	6	5	4	3	2	1	0
Read	TIE	TCIE	RIE	ILIE	TE	RE	RWU	SBK
Write								
Reset	0	0	0	0	0	0	0	0

- Interrupt Enables
 - TIE: Interrupt when Transmit Data Register is empty
 - TCIE: Interrupt when transmission completes
 - RIE: Interrupt when receiver has data ready
- Module Enables
 - TE: Transmitter enable
 - RE: Receiver enable
- Other
 - RWU: Put receiver in standby mode, will wake up when condition occurs
 - SBK: Send a break character (all zeroes)

UART Data Register (UART_D)



- 8-bit data register
- A write-to UARTx_D writes to the transmit buffer
- A read-from UARTx_D reads from the receive buffer

UART Status Register 1 (UART_S1)

Bit	7	6	5	4	3	2	1	0
Read	TDRE	TC	RDRF	IDLE	OR	NF	FE	PF
Write				w1c	w1c	w1c	w1c	w1c
Reset	1	1	0	0	0	0	0	0

- TDRE: Transmit data register empty, can write more data to data register
- TC: Transmission complete.
- RDRF: Receiver data register full, can read data from data register
- IDLE: UART receive line has been idle for one full character time
- OR: Receive overrun. Received data has overwritten previous data in receive buffer
- NF: Noise flag. Receiver data bit samples don't agree.
- FE: Framing error. Received 0 for a stop bit, expected 1.
- PF: Parity error. Incorrect parity received.

UART Status Register 2 (UARTx_S2)

Bit	7	6	5	4	3	2	1	0
Read	LBDIF	RXEDGIF	MSBF	RXINV	RWUID	BRK13	LBKDE	RAF
Write								
Reset	0	0	0	0	0	0	0	0

- LBDIF: LIN break detect interrupt flag
- RXEDGIF: Active edge on receive pin detected
- **MSBF: Send MSB first. Should be 0 for RS232**
- **RXINV: Invert received signals (data, start, stop, etc.)**
- RWUID: Set idle bit upon wakeup?
- BRK13: Set break character to 13 bits long (not 10)
- LBKDE: LIN break character time.
- RAF: Receiver is actively receiving data (not idle line)

Steps for Programming UART

- Enables clock gating for UART and Port.
- Disables UART receiver and transmitter to allow access to control registers.
- Selects oscillator clock source for baud rate generator.
- Connects Pin to UART_Rx and UART_Tx respectively.
- Sets the serial baud rate divider value.
- Sets the oversampling factor.
- Configures for eight data bits, no parity and one stop bit.
- Enables the transmitter and receiver.

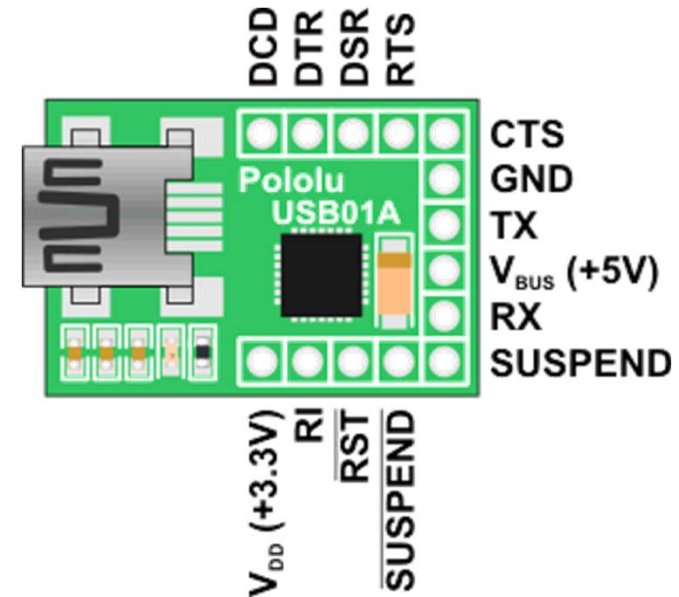
Software for Interrupt-Driven Serial Comm.

- Use interrupts
- First, initialize peripheral to generate interrupts
- Second, create single ISR with three sections corresponding to cause of interrupt
 - Transmitter
 - Receiver
 - Error



USB to UART Interface

- PCs haven't had external asynchronous serial interfaces for a while, so how do we communicate with a UART?

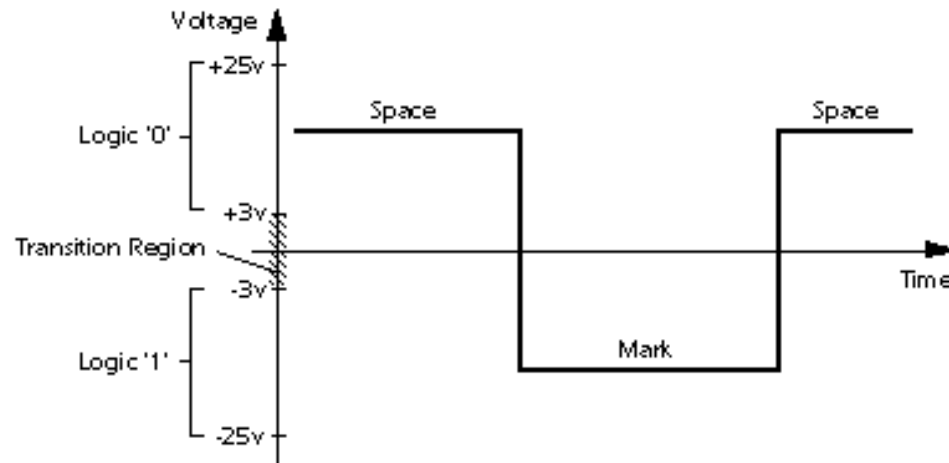


- USB to UART interface
 - USB connection to PC
 - Logic level (0-3.3V) to microcontroller's UART (not RS232 voltage levels)
- USB01A USB to serial adaptor
 - <http://www.pololu.com/catalog/product/391>
 - Can also supply 5 V, 3.3 V from USB

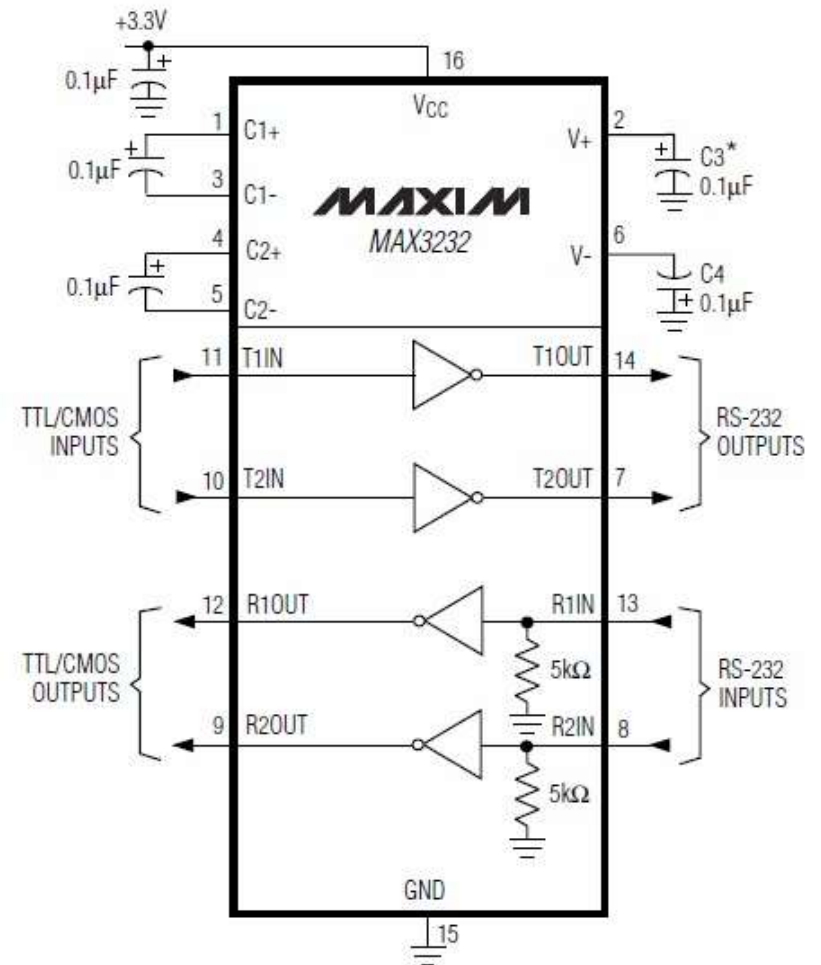
Building on Asynchronous Comm.

- Asynchronous communication is useful but runs into some problems when applying it to some applications
- Problem #1
 - Logic-level signals (0 to 1.65 V, 1.65 V to 3.3 V) are sensitive to noise and signal degradation
- Problem #2
 - Point-to-point topology does not support a large number of nodes well
 - Need a dedicated wire to send information from one device to another
 - Need a UART channel for each device the MCU needs to talk to
 - Single transmitter, single receiver per data wire

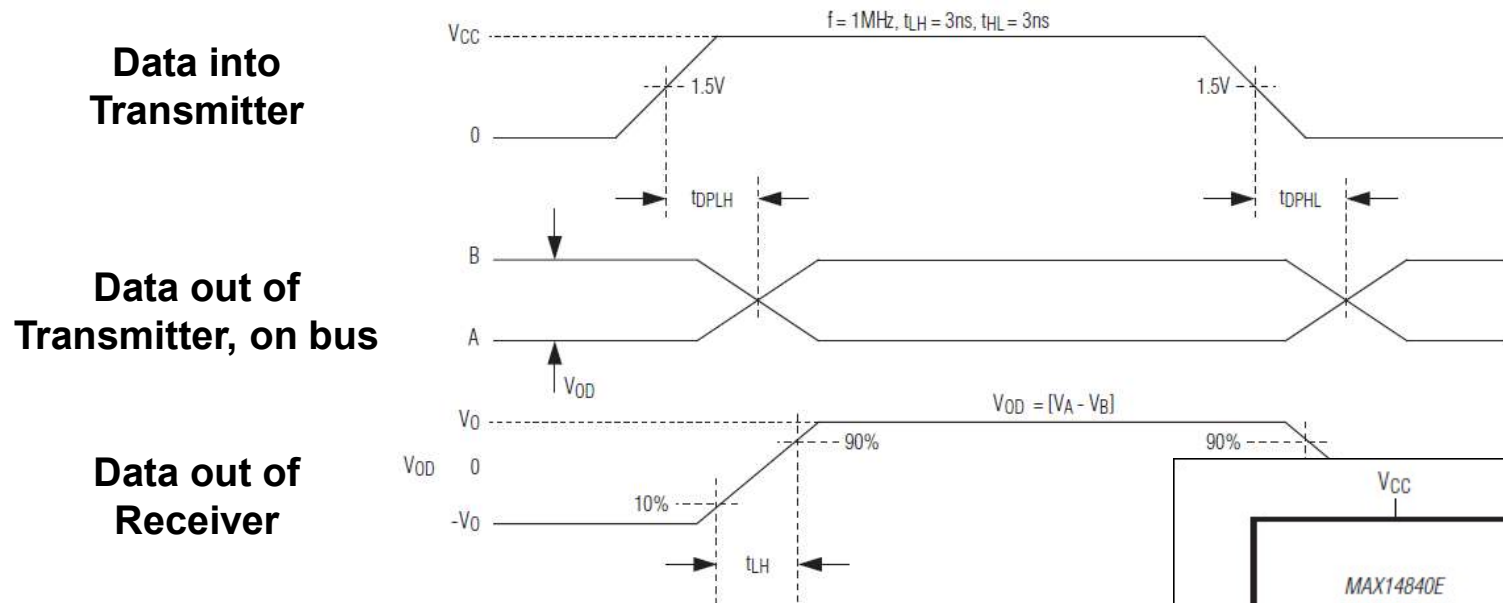
Solution to Noise: Higher Voltages



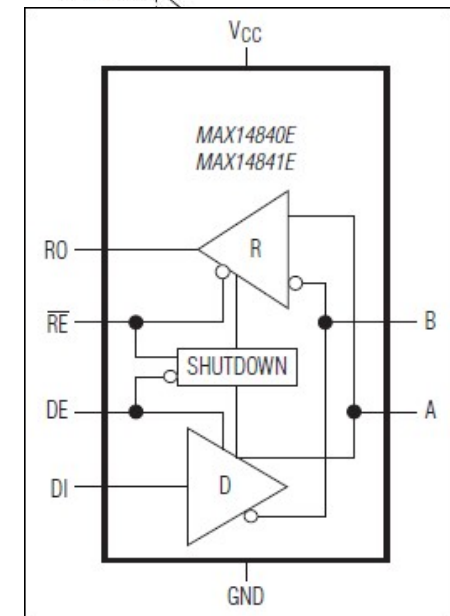
- Use higher voltages to improve noise margin:
+3 to +15 V, -3 to -15 V
- Example IC (Maxim MAX3232) uses charge pumps to generate higher voltages from 3.3V supply rail



Solution to Noise: Differential Signaling



- Use differential signaling
 - Send two signals: Buffered data (A), buffered complement of data (B)
 - Receiver compares the two signals to determine if data is a one ($A > B$) or a zero ($B > A$)



Solutions to Poor Scaling

- Approaches
 - Allow one transmitter to drive multiple receivers (multi-drop)
 - Connect all transmitters and all receivers to same data line (multi-point network). Need to add a medium access control technique so all nodes can share the wire
- Example Protocols
 - RS-232: higher voltages, point-to-point
 - RS-422: higher voltages, differential data transmission, multi-drop
 - RS-485: higher voltages, multi-point

Outline

- Serial communication Concept
- Software Architecture for Handling asynchronous Communication
- SPI Communications
- I2C Communications
- Asynchronous serial (UART) Communications
- **Protocol Comparison**
- Summary



Factors to Consider

- How fast can the data get through?
 - Depends on raw bit rate, protocol overhead in packet
- How many hardware signals do we need?
 - May need clock line, chip select lines, etc.
- How do we connect multiple devices (topology)?
 - Dedicated link and hardware per device - point-to-point
 - One bus for master transmit/slave receive, one bus for slave transmit/master receive
 - All transmitters and receivers connected to same bus – multi-point
- How do we address a target device?
 - Discrete hardware signal (chip select line)
 - Address embedded in packet, decoded internally by receiver
- How do these factors change as we add more devices?

Protocol Trade-Offs

Protocol	Speed	Signals Req. for Bidirectional Communication with N devices	Device Addressing	Topology
UART (Point to Point)	Fast – Tens of Mbit/s	2*N (TxD, RxD)	None	Point-to-point full duplex
UART (Multi-drop)	Fast – Tens of Mbit/s	2 (TxD, RxD)	Added by user in software	Multi-drop
SPI	Fast – Tens of Mbit/s	3+N for SCLK, MOSI, MISO, and one SS per device	Hardware chip select signal per device	Multi-point full-duplex, multi-drop half-duplex buses
I ² C	Moderate – 100 kbit/s, 400 kbit/s, 1 Mbit/s, 3.4 Mbit/s. Packet overhead.	2: SCL, SDA	In packet	Multi-point half-duplex bus

Summary

- Motivation for communicating information serially and the core issues that must be tackled to do so.
- Helpful development tools and software structures.
- Three different types of communication protocols (asynchronous serial, synchronous serial, and I2C) and how to implement them using the peripherals of the MCU.